



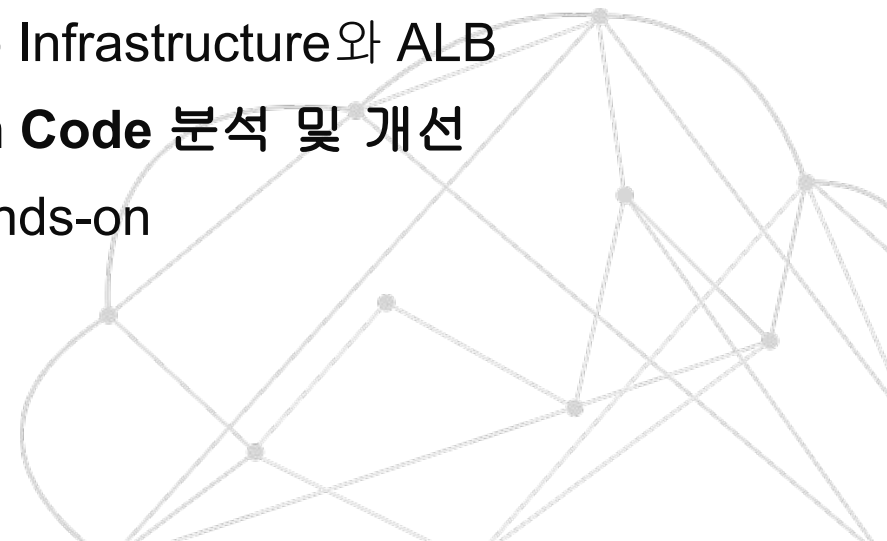
Terraform IaC Essential Hands-on

Bespin Academy 류승석
seungsuk.ryoo@bespinglobal.com
2026.08.28(금)

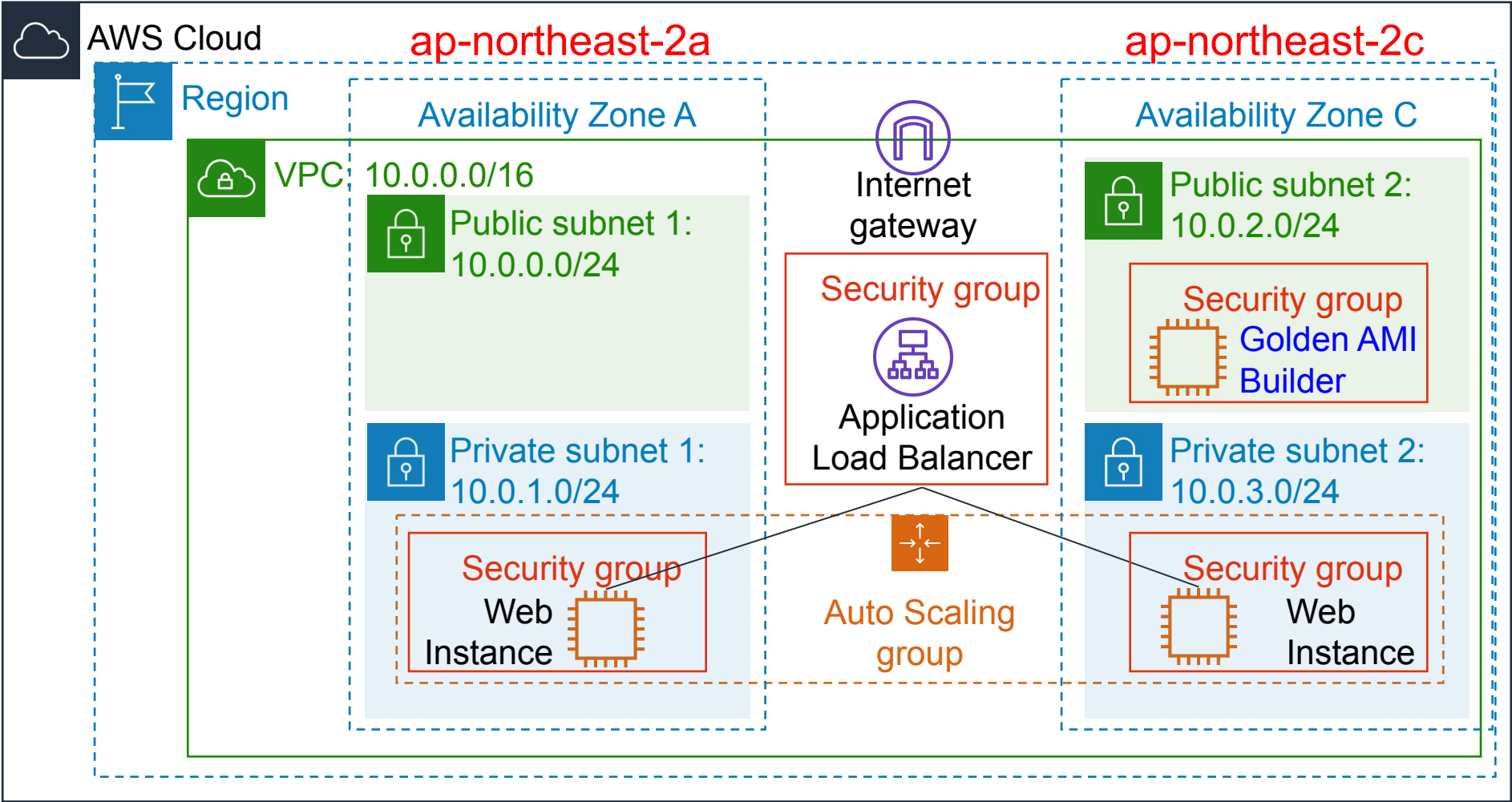
Terraform IaC Essential Hands-on

CHAPTER

1. Terraform을 이용한 Infrastructure as Code 이해
2. IAM 계정 생성 및 Default VPC 확인
3. Custom VPC 기반 Highly Available Web Architecture 구축
 - Lab 1. Development Environment
 - Lab 2. Terraform Basics와 Network
 - Lab 3. Golden AMI
 - Lab 4. Private Web Infrastructure와 ALB
4. AI를 활용한 Terraform Code 분석 및 개선
 - Lab 5. Kiro CLI Hands-on
5. 리소스 삭제



Final Architecture



퍼블릭 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬

0.0.0.0/0	IGW
-----------	-----



Terraform을 이용한 Infrastructure as Code 이해

Terraform을 이용한 Infrastructure as Code 이해

코드형 인프라 (Infrastructure as Code, IaC)란 무엇인가요 ?

- IaC는 **기계가 읽을 수 있는 코드**로 인프라를 정의·관리하는 방식
- 인프라를 작성 (**Write**) → 배포 (**Provision**) → 업데이트 (**Update**) → 삭제 (**Destroy**)
모두 **코드를 통해 자동화**
- 문서가 아니라, **실행 가능한 인프라 설계서**
→ *“Think of IaC as executable documentation.”*

왜 IaC가 필요한가요 ?

전통적 문제점

- 콘솔에서 클릭으로 인프라 구성 → **느림** + 사람마다 방식이 달라 구성 **불일치**
- 동일 환경(dev/stage/prod) 구성 시 반복 작업 필요
- 인프라 변경·이력 관리 어려움

IaC의 해결책

- 인프라를 **코드화** → **표준화** → **자동화**
- 클릭 아닌 코드로 구성하므로 **재현성, 일관성, 버전관리** 가능
- DevOps·클라우드 네이티브 시대의 필수 기술

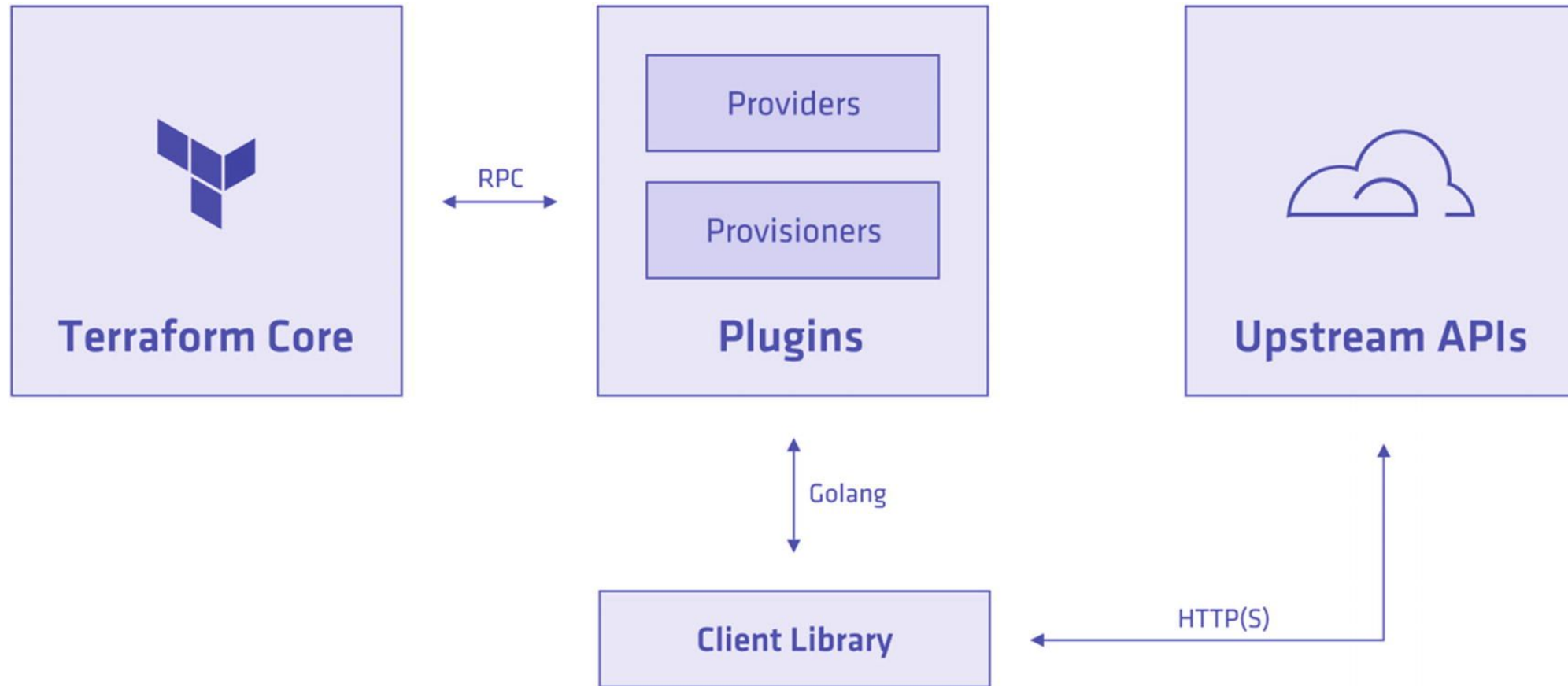
테라폼 (Terraform) 이란 ?

- [Terraform](#)은 Hashicorp에서 개발한 오픈소스 **Infrastructure as Code(IaC)** 도구
- 적은 코드로 클라우드 인프라를 생성·변경·삭제·버전관리
- AWS, Azure, GCP, Kubernetes 등 멀티클라우드 /멀티플랫폼 지원
- 선언형(Declarative) 방식 → “원하는 상태(Desired State)”만 정의하면 Terraform이 자동으로 맞춰줌



Terraform Architecture

- Generic Terraform architecture



Deep-Dive Terraform on Azure, Ritesh Modi, Apress. [Fig1](#)

Terraform Architecture (계속)

1. Terraform Core

- 사용자가 작성한 HCL 코드를 읽고 **Plan** → **Apply** → **State 관리**를 수행하는 핵심 엔진
- 실제 클라우드 API 호출은 직접 하지 않고, Provider 플러그인에게 위임

2. Plugins (Providers & Provisioners)

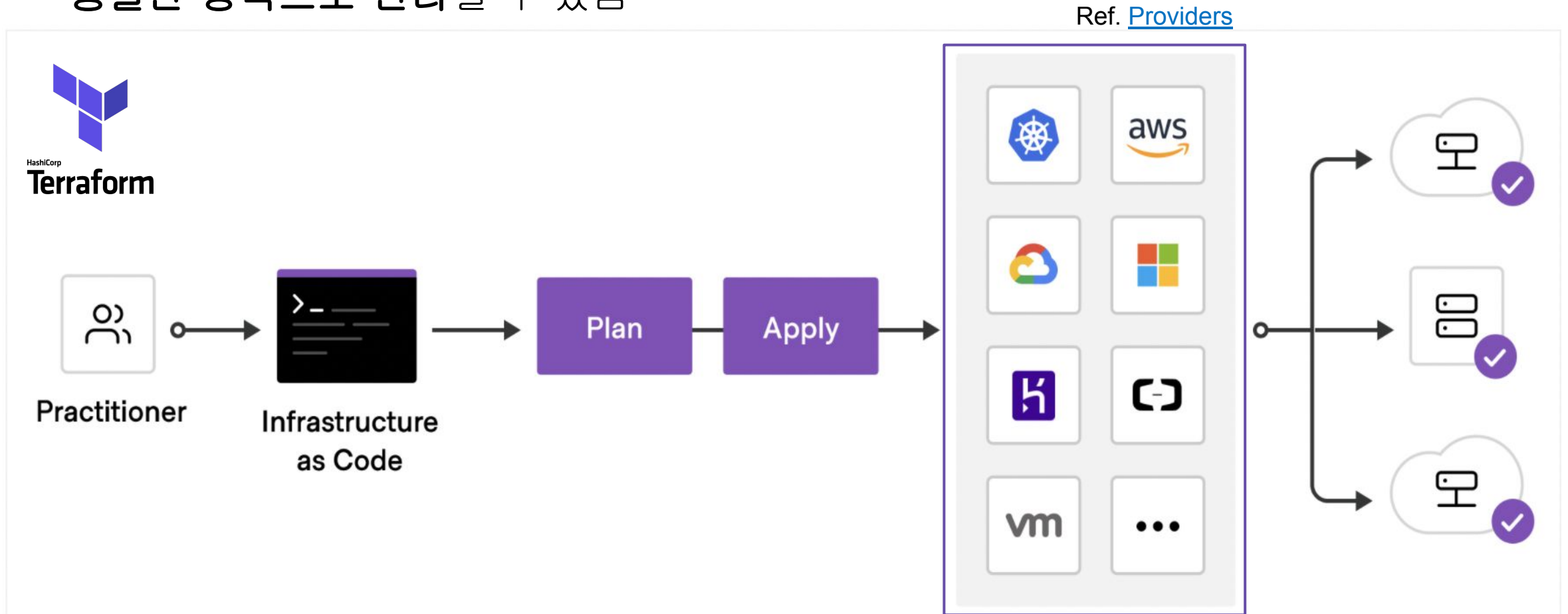
- **Provider**: AWS, GCP, Azure, Kubernetes 등 클라우드/서비스 API와 통신하는 컴포넌트 (AWS Provider, Kubernetes Provider 등)
- Terraform Core ↔ Provider 간 통신은 **RPC(Remote Procedure Call)**
- Provider 내부에서는 각 CSP에서 제공하는 **Client Library**(ex. AWS SDK for Go)를 사용하여 실제 API 호출 수행

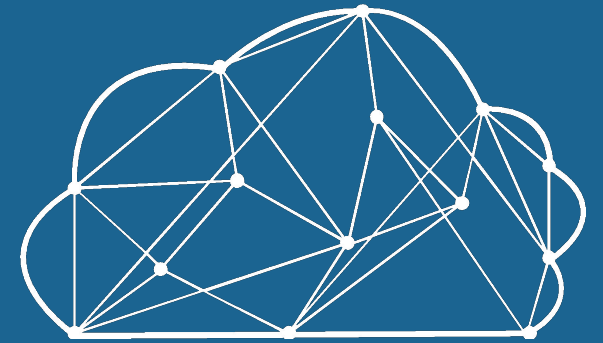
3. Upstream APIs (Cloud Services)

- Provider가 각 클라우드 API(AWS EC2, VPC, S3 등)를 HTTPS로 호출하여 리소스를 생성·변경·삭제

Multi-Cloud 지원

- 사용자는 코드를 한 번 작성하고, Plan → Apply 과정으로 여러 클라우드 인프라를 동일한 방식으로 관리할 수 있음



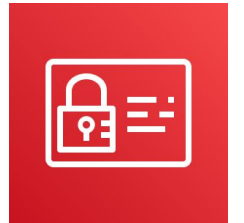


IAM 계정 생성 및 Default VPC 확인

IAM 계정 생성 및 Default VPC 확인

AWS Identity and Access Management (IAM)

- AWS Identity and Access Management([IAM](#))는 Amazon Web Services(AWS) 고객이 AWS에서 사용자와 사용자 권한을 관리할 때 사용할 수 있는 서비스입니다.
- IAM을 사용하여 AWS 리소스에 대한 액세스 관리 –
 - 리소스는 사용자가 작업을 수행할 수 있는 AWS 계정의 엔터티입니다.
 - 리소스의 예로는 Amazon EC2 인스턴스 또는 Amazon S3 버킷이 있습니다.
- 예 – Amazon EC2 인스턴스를 종료할 수 있는 사용자 제어
- 세분화된 액세스 권한 정의 –
 - 리소스에 액세스할 수 있는 사용자
 - 액세스할 수 있는 리소스와 사용자가 리소스에 수행할 수 있는 작업
 - 리소스에 액세스하는 방법
- IAM은 AWS 계정에 무료로 제공되는 기능입니다.



AWS Identity and
Access Management
(IAM)

IAM: 필수 구성 요소



- IAM 사용자

- AWS 계정으로 **인증할 수 있는 사람** 또는 애플리케이션입니다.



- IAM 그룹

- 동일한 권한 부여를 허락받은 **IAM 사용자의 모음**입니다.



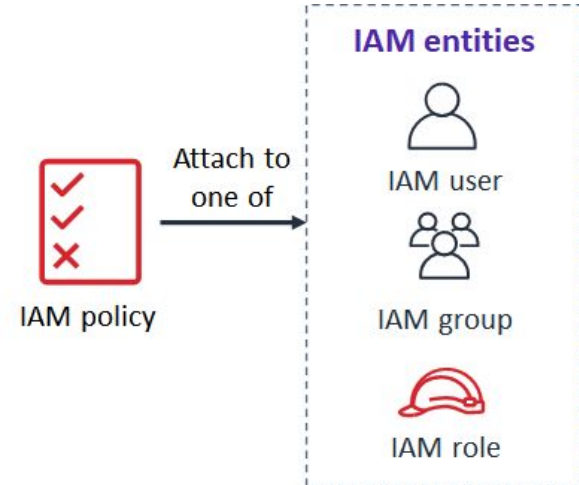
- IAM 정책

- 액세스할 수 있는 리소스와 각 리소스에 대한 **액세스 수준을 정의하는 문서**입니다.

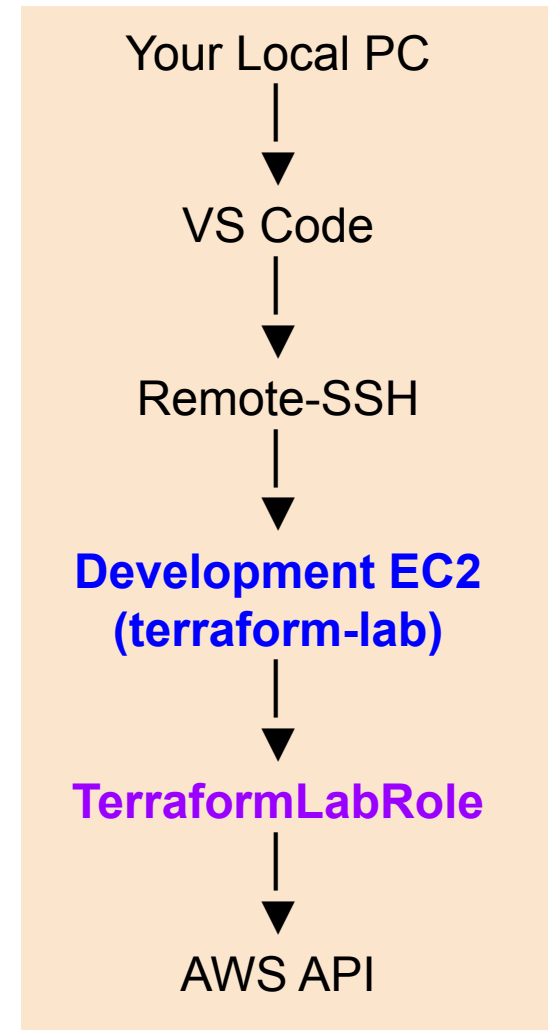
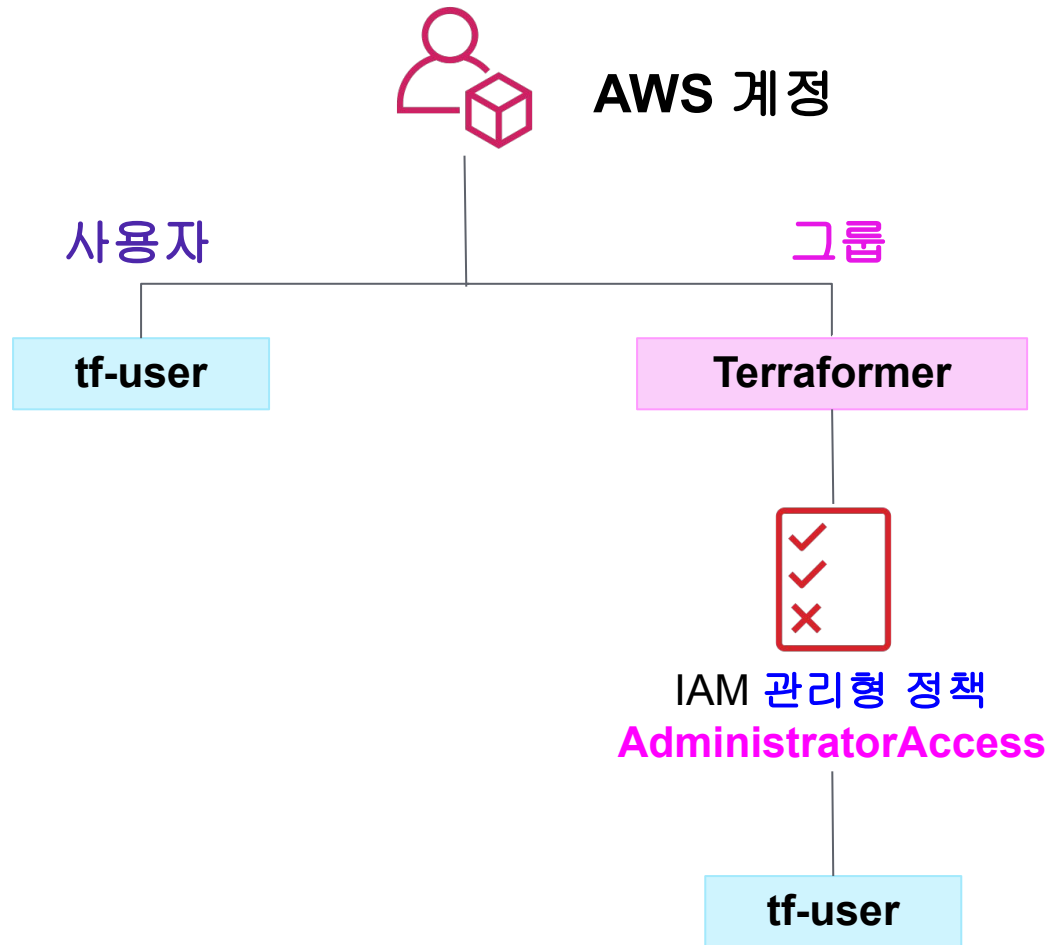


- IAM 역할

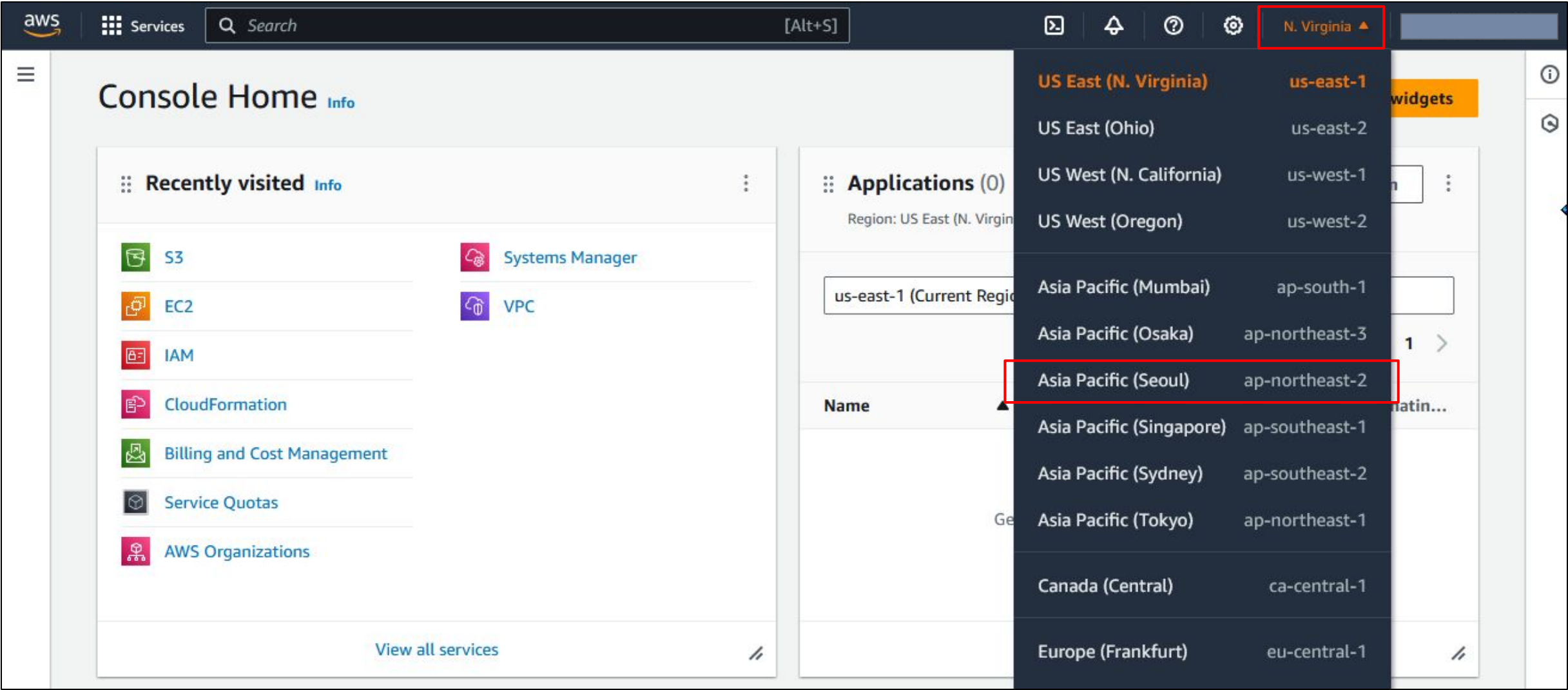
- **AWS 서비스** 요청을 위한 **권한 세트를 부여**하는 **유용**한 메커니즘입니다.



이 실습에서 다루는 주제 (계속)



AWS Management Console 로그인 (서울 리전)

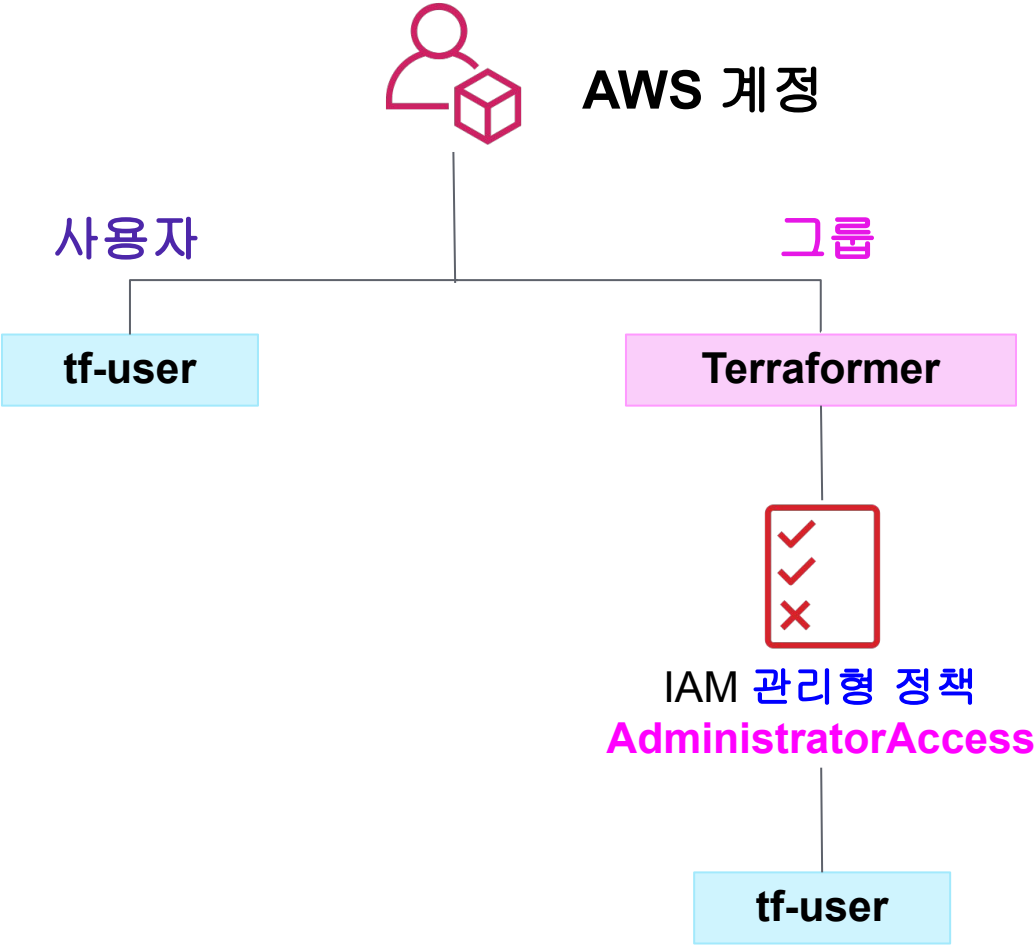


Task

1. Create IAM group and user
2. AWS Management Console Login
3. Assign MFA device
4. Default VPC



Create IAM group and user



Create IAM group and user (계속)

1. 테라폼 개발을 위한 IAM 그룹과 사용자 계정을 생성하도록 하겠습니다.
(적절한 권한의 IAM 계정을 사용하시는 분은 이 부분을 건너 뛰어도 됩니다.)
2. Services -> IAM -> User groups -> **Create group**을 클릭하여 그룹을 추가합니다.
3. IAM 그룹 생성
 - User group name: [Terraformer](#)
 - Add users to the group: 여기서는 사용자를 추가하지 않음
 - Attach permissions policies: [AdministratorAccess](#) 정책을 찾아서 체크(선택)
4. 우측 하단의 **Create user group** 버튼을 클릭하여 그룹 생성을 완료

Create IAM group and user (계속)

5. Services -> IAM -> Users -> Create user를 클릭하여 사용자를 추가
6. IAM 사용자 생성
 - User name: [tf-user](#)
 - Provide user access to the AWS Management Console - optional: [체크](#)
 - Console password: [Autogenerated password](#) 선택 (기본값)
 - Users must create a new password at next sign-in - Recommended: [체크](#) (기본값)
7. 우측 하단의 **Next** 버튼 클릭
8. Add user to group에서 [Terraformer](#) 그룹을 선택
9. 우측 하단의 **Next** 버튼 클릭
10. Review and create에서 우측 하단의 **Create user** 버튼 클릭
11. Retrieve password에서 **Download .csv file** 버튼을 클릭하여 인증 정보를 저장 (파일명: [tf-user_credentials.csv](#))
12. 우측 하단의 **Return to users list** 버튼을 클릭하여 사용자 생성을 완료합니다.
13. 다운받은 .csv 파일을 열어보면 다음과 같습니다. (다음 페이지 참고)

Create IAM group and user (계속)

	A	B	C	D	E	F
1	User name	Password	Console sign-in URL			
2	tf-user					
3						

< >

tf-user_credentials

+

:

AWS 관리 콘솔
로그인 ID

AWS 관리 콘솔
로그인 PW

AWS 관리 콘솔
IAM 로그인 창 URL

AWS Management Console Login

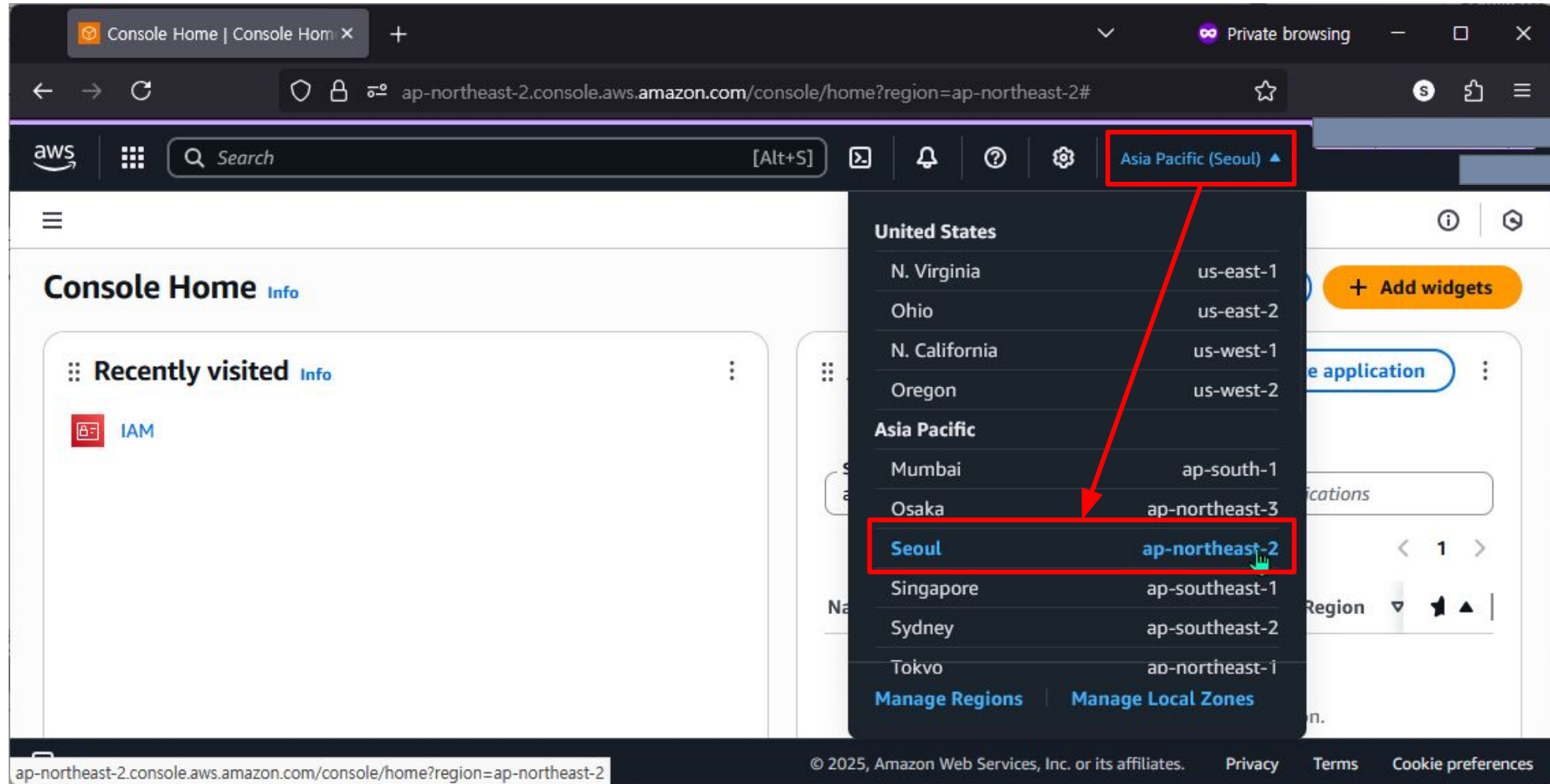
1. 신규로 생성한 **tf-user** 사용자로 로그인 하겠습니다.
2. **tf-user_credentials.csv** 파일을 열어 **Console sign-in URL**을 복사 → 웹 브라우저의 주소창에 붙여넣기 → IAM 로그인 창으로 이동합니다.
3. 계정 ID(12자리)는 자동으로 값이 입력됩니다.
4. **tf-user_credentials.csv** 파일의 내용을 참고하여 **User name(사용자 이름)**과 **Password(암호)**를 입력하고 로그인 버튼을 클릭합니다.
5. 최초 로그인 시 패스워드를 변경하도록 설정하였으므로, 아래와 같이 패스워드를 갱신합니다.
 - 이전 비밀번호: 앞에서 입력한 값 – **tf-user_credentials.csv** 파일의 **Password** 값 입력
 - 새 비밀번호: 신규 비밀번호 입력
 - 새 비밀번호 재입력: 위에서 입력한 신규 비밀번호 재입력
 - **비밀번호 변경 확인** 버튼 클릭

Assign MFA device

1. Services → IAM → **Add MFA** 버튼 클릭 (또는 IAM → Users → tf-user → Security credentials 탭에서 **Assign MFA device** 버튼 클릭)
2. Select MFA device 에서
 - Device name: **tf-user** (또는 개인 이니셜, **ssryoo** 등)
 - MFA device: **Authenticator app** 선택 (**중요!**)
 - 우측 하단의 **Next** 버튼 클릭
3. Set up device 에서
 - 개인 스마트 폰에 Google Authenticator 등의 **OTP앱 설치** (이미 설치되었으면 해당 앱 실행)
 - 웹브라우저의 **Show QR code** 링크를 클릭하여 QR code를 생성
 - 스마트 폰의 Authenticator 앱에서 **+ 버튼**을 클릭하여 QR code를 스캔, OTP 등록
 - **MFA code 1**과 **MFA code 2**에 연속적으로 바뀌는 OTP 숫자를 **순서대로** 입력(6자리)
 - **Add MFA** 버튼을 눌러서 MFA 등록
4. 성공적으로 등록(assigned)되면, 우측 상단의 계정명 클릭 → **Sign out** → 다시 로그인 을 수행 (tf-user_credentials.csv)하여 MFA 인증(멀티 팩터 인증)이 잘 수행되는지 확인합니다.

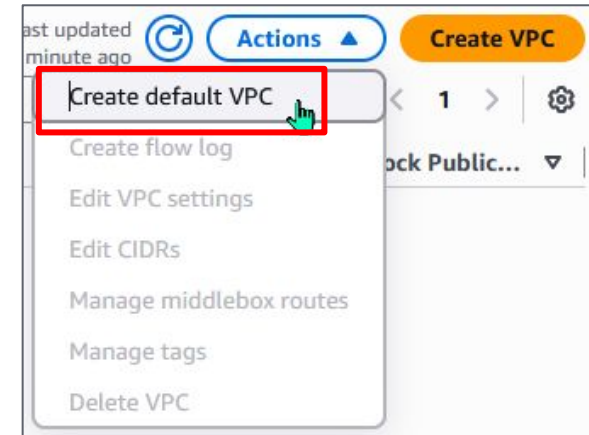
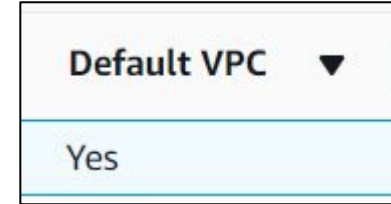
Default VPC

1. 아래와 같이 클릭하여 **서울 리전**으로 이동합니다. (이미 서울 리전이면 선택하지 않아도 됨)



Default VPC (계속)

2. Services -> VPC -> **Your VPCs**를 클릭하여 생성된 VPC를 살펴봅니다.
3. 실습을 위하여 우측의 그림과 같이 **Default VPC**가 **Yes**인 VPC가 있어야 합니다.
4. 만일 **Default VPC**가 없다면, 아래의 과정을 통하여 신규로 생성합니다.
 - 우측 상단의 **Actions** 선택 → **Create default VPC**
 - 생성한 VPC 이름을 **Default VPC**로 설정
5. **Default VPC**의 서브넷이 잘 생성되어 있는지 확인합니다.
 - 좌측 메뉴에서 **Subnets** 선택
 - 좌측 메뉴 상단에 **Filter by VPC:** 에서 **Default VPC** 선택
 - 아래의 그림과 같이, 모든 가용영역의 서브넷이 존재해야 합니다.



Subnets (4) Info

Last updated 2 minutes ago

Actions

Create subnet

Find subnets by attribute or tag

< 1 >

	Name	Subnet ID	State	VPC	IPv4 CIDR	Avail...	Availability Zone
☐	-	subnet-0e55e5...	Available	vpc-0a8d523b774776d73 De...	172.31.0.0/20	4091	apne2-az1 (ap-northeast-2a)
☐	-	subnet-025fb2...	Available	vpc-0a8d523b774776d73 De...	172.31.16.0/20	4091	apne2-az2 (ap-northeast-2b)
☐	-	subnet-04d0cb...	Available	vpc-0a8d523b774776d73 De...	172.31.32.0/20	4091	apne2-az3 (ap-northeast-2c)
☐	-	subnet-0e5cac...	Available	vpc-0a8d523b774776d73 De...	172.31.48.0/20	4091	apne2-az4 (ap-northeast-2d)

Default VPC (계속)

6. 좌측 메뉴에서 Internet gateways 선택 → IGW가 정상적으로 생성/연결(Attached)되었는지 확인

VPC > Internet gateways

VPC dashboard

EC2 Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Internet gateways (1) Info

Find internet gateways by attribute or tag

Name

Internet gateway ID

State

VPC ID

-

[igw-05dde1576d2dd5371](#)

✓ Attached

[vpc-0a8d523b774776d73](#) | Default VPC

http://www.bespinglobal.com
Copyright © 2026 BESPIN GLOBAL, Inc. All rights reserved | Confidential

28

Default VPC (계속)

- 7. 좌측 메뉴에서 Route tables 선택 → Route table 클릭
 - Routes 탭 → IGW Active 확인
 - Subnet associations 탭 → 4개의 서브넷과 연결되어 있는지 확인

rtb-07be42381384be3ed

Details

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Both

Edit routes

Filter routes

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-05dde1576d2dd5371	Active	No	
172.31.0.0/16	local	Active	No	

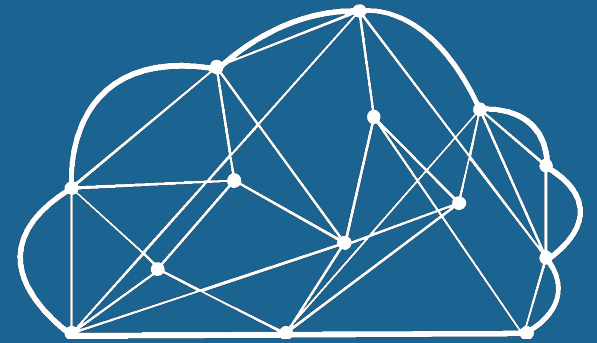
Subnets without explicit associations (4)

Edit subnet associations

The following subnets have not been explicitly associated with any route tables and are therefore associated with the main route table:

Find subnet association

Name	Subnet ID	IPv4 CIDR	IPv6 CIDR
-	subnet-0e55e5dffa65ca643	172.31.0.0/20	-
-	subnet-025fb207eac6b2ac6	172.31.16.0/20	-
-	subnet-04d0cbdbc3a0ef02e	172.31.32.0/20	-
-	subnet-0e5cacd9755ae7460	172.31.48.0/20	-



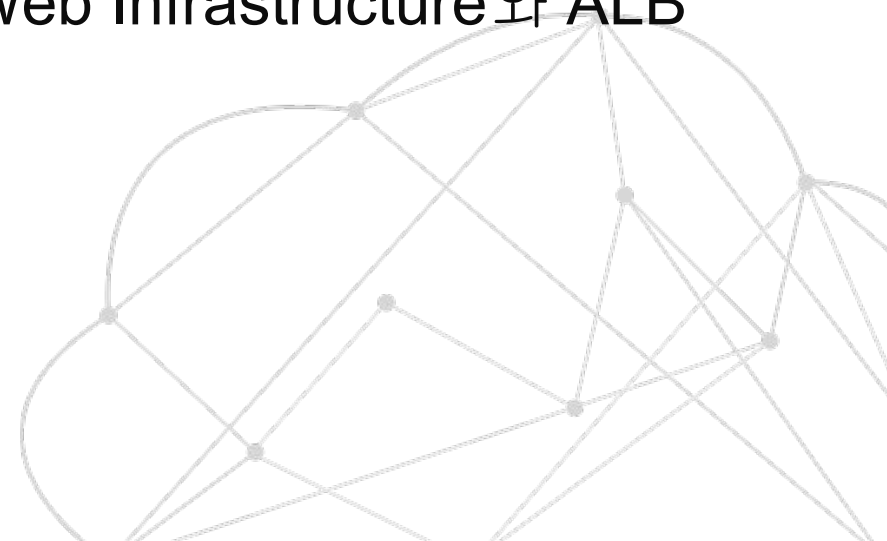
Custom VPC 기반 Highly Available Web Architecture 구축

Custom VPC 기반 Highly Available Web Architecture 구축

SUB CHAPTER

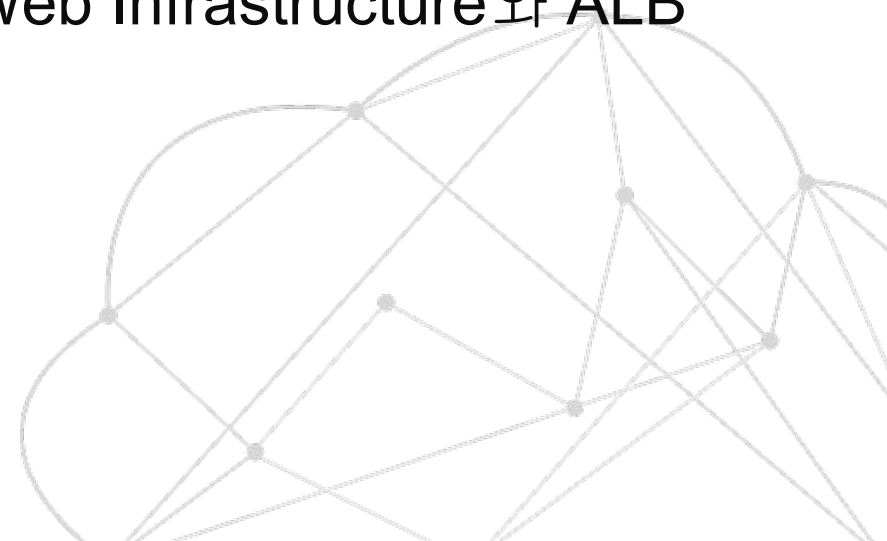
1. Custom VPC 기반 Highly Available Web Architecture 구축

- Lab 1. Development Environment
- Lab 2. Terraform Basics와 Network
- Lab 3. Golden AMI
- Lab 4. Private Web Infrastructure와 ALB



SUB CHAPTER

1. Custom VPC 기반 Highly Available Web Architecture 구축
 - Lab 1. Development Environment
 - Lab 2. Terraform Basics와 Network
 - Lab 3. Golden AMI
 - Lab 4. Private Web Infrastructure와 ALB



Lab 1. Development Environment

Task

1. Visual Studio Code 설치
2. Ubuntu Development EC2 생성
3. TerraformLabRole 생성과 EC2 연결
4. VS Code Remote-SSH 연결
5. Repository Clone과 환경 자동화
6. Development Environment Checkpoint



Setup VS Code

1. <https://code.visualstudio.com/download> 사이트에 접속하여 User Installer 다운로드 및 설치

Download Visual Studio Code

Free and built on open source. AI agents, Git, debugging, and extensions included.



↓ Windows

Windows 10, 11

User Installer

x64Arm64

System Installer

x64Arm64

.zip

x64Arm64

CLI

x64Arm64



↓ .deb

Debian, Ubuntu

.deb

x64Arm32Arm64

.rpm

x64Arm32Arm64

.tar.gz

x64Arm32Arm64

Snap

Snap Store

CLI

x64Arm32Arm64



↓ Mac

macOS 12.0+

.dmg

Intel chipApple siliconUniversal

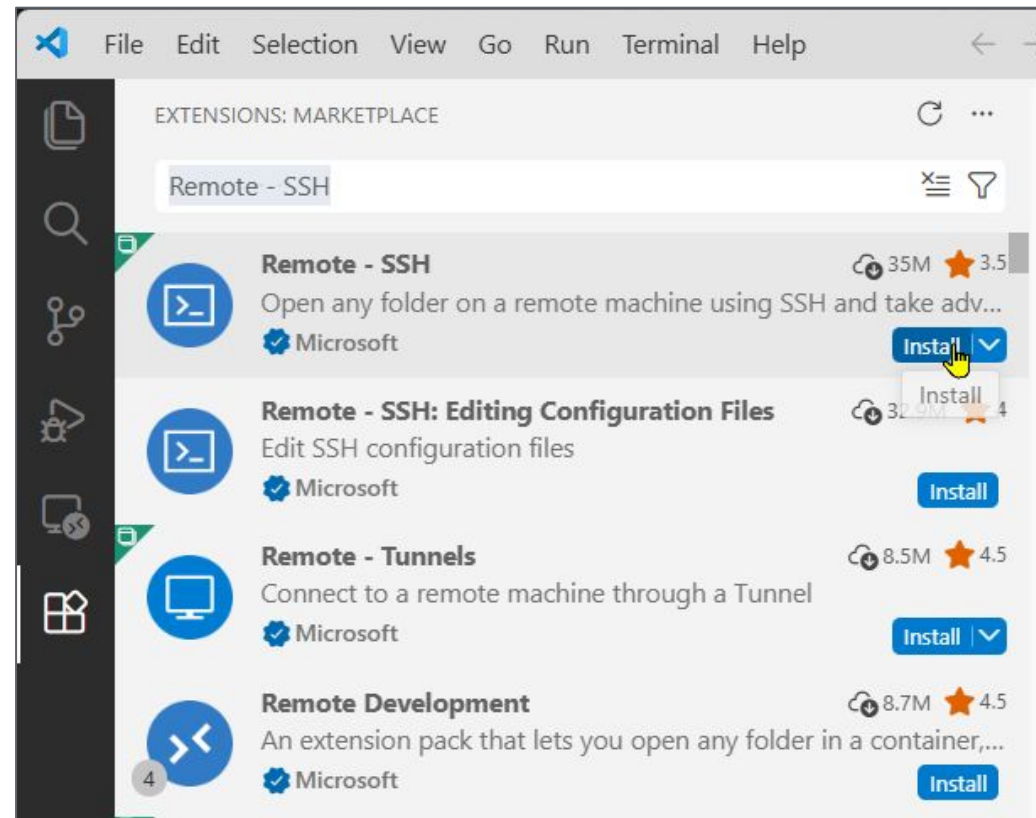
CLI

Intel chipApple silicon

VS Code Remote SSH를 이용한 EC2 원격 연결

Remote - SSH Extension 설치

1. VS Code 실행 → 왼쪽 Extensions 선택
2. 검색 → **Remote - SSH** 설치 (제작자: Microsoft)



VS Code Remote SSH를 이용한 EC2 원격 연결 (계속)

SSH Key 준비

1. tfkey.pem 다운로드
2. (Windows 기준)
 - C:\Users\<계정명>\.ssh 폴더 생성
 - 예) C:\Users\Administrator\.ssh\
3. tfkey.pem 파일 복사

```
C:\Users\Administrator\.ssh>
C:\Users\Administrator\.ssh>dir labkey.pem
C 드라이브의 볼륨에는 이름이 없습니다.
볼륨 일련 번호: DE0C-069A

C:\Users\Administrator\.ssh 디렉터리

2026-06-05 오후 02:10                1,674 labkey.pem
                        1개 파일                1,674 바이트
                        0개 디렉터리  203,280,130,048 바이트 남음

C:\Users\Administrator\.ssh>
```

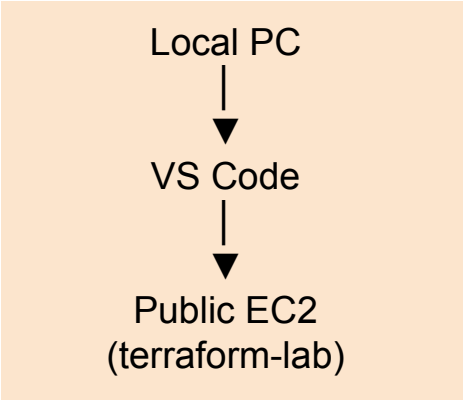
VS Code Remote SSH를 이용한 EC2 원격 연결 (계속)

SSH 설정 추가 및 파일 저장

Extension: Remote - SSH config X

C: > Users > Administrator > .ssh > config

```
1 # Read more about SSH config files: https://linux.die.net/man/5/ssh_config
2 Host aws-lab
3     HostName 54.180.61.221
4     User ubuntu
5     IdentityFile C:\Users\Administrator\.ssh\labkey.pem
6
```



● [참고] 주요 옵션

옵션	설명	예시
Host	SSH 접속 시 사용할 별칭(Alias)	aws-lab
HostName	접속 대상 서버의 IP 또는 DNS	43.201.100.100
User	로그인 계정	ubuntu
IdentityFile	SSH Private Key 경로	~/.ssh/labkey.pem

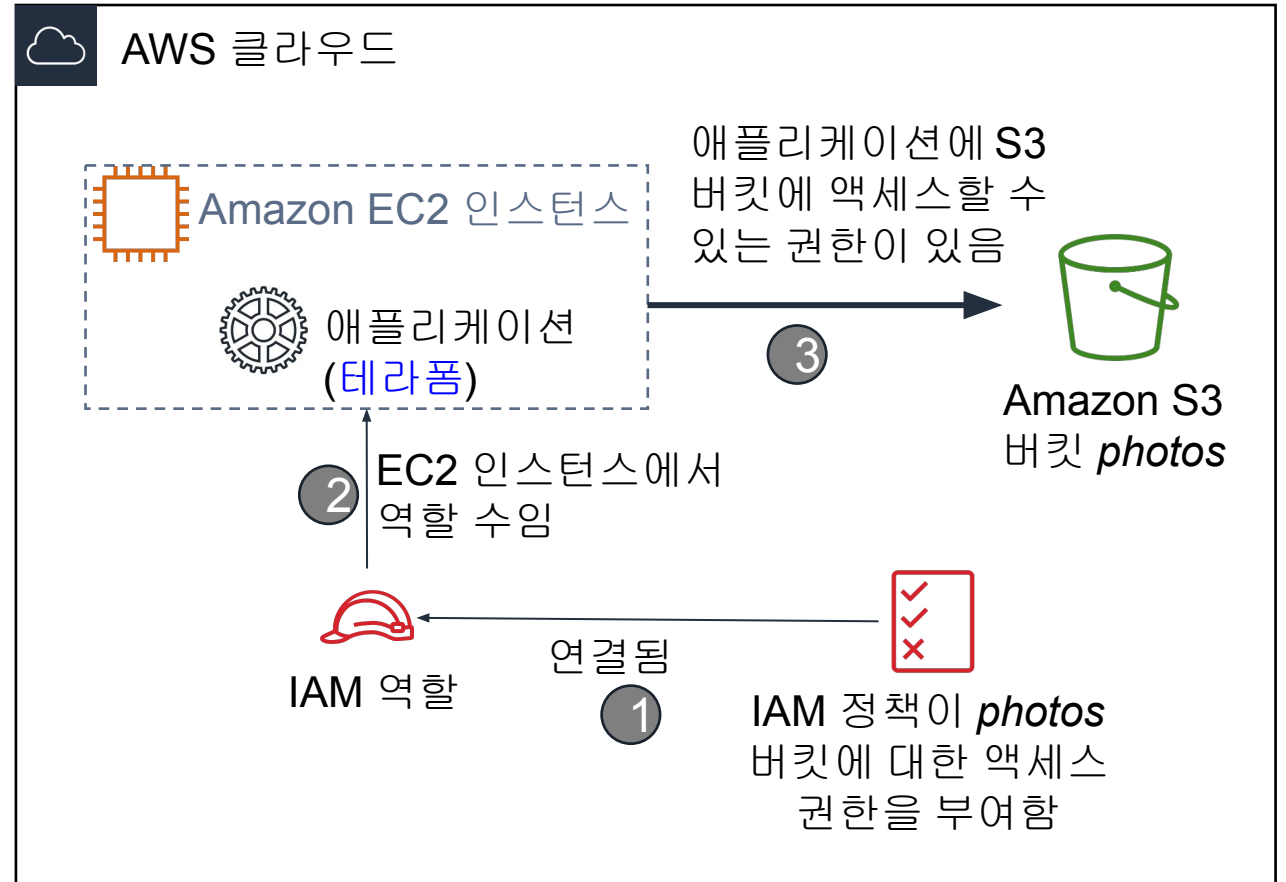
TerraformLabRole 생성과 Ubuntu Development EC2 연결

시나리오:

- EC2 인스턴스에서 실행되는 애플리케이션(테라폼)에 S3 버킷에 대한 액세스 권한이 필요

솔루션:

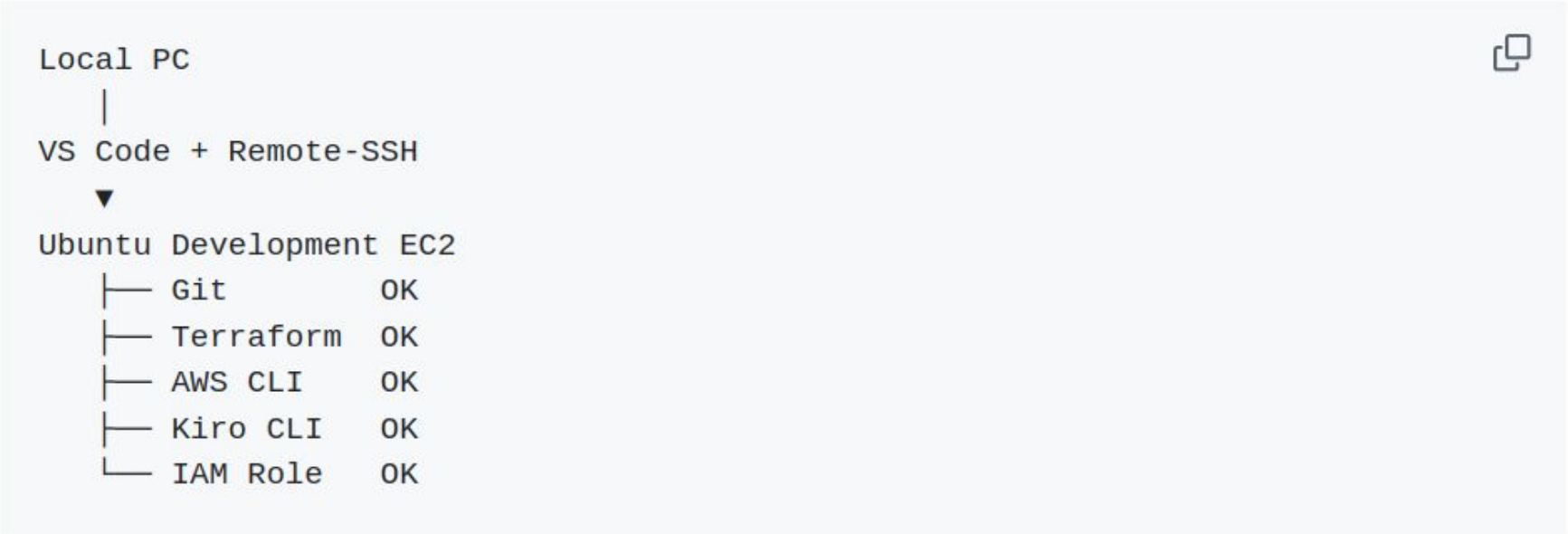
- S3 버킷에 대한 액세스 권한을 부여하는 IAM 정책 정의
- 정책을 역할에 연결
- EC2 인스턴스가 이 역할을 수임하는 것을 허용



“이 원리를 TerraformLabRole에 적용합니다.”

Lab 1. Development Environment

1교시 종료 목표:



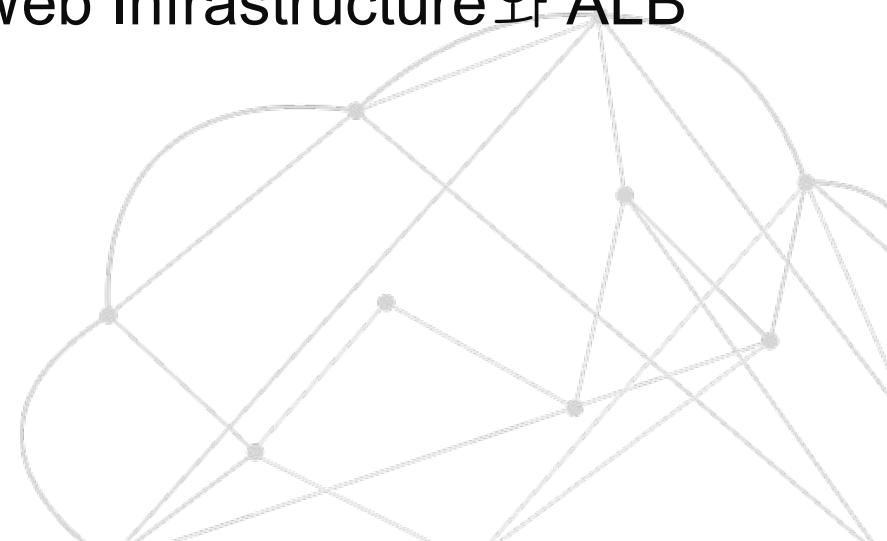
1-1. Visual Studio Code

교육생 PC에 [Visual Studio Code](#)를 설치합니다. Windows 10/11, macOS와 Ubuntu Linux의 자세

SUB CHAPTER

1. Custom VPC 기반 Highly Available Web Architecture 구축

- Lab 1. Development Environment
- Lab 2. Terraform Basics와 Network
- Lab 3. Golden AMI
- Lab 4. Private Web Infrastructure와 ALB



Lab 2. Terraform Basics와 Network

Basic CLI Features

- To view a list of the commands available in your current Terraform version, run `terraform` with no additional arguments:

```
$ terraform
```

```
Usage: terraform [global options] <subcommand> [args]
```

```
...
```

```
Main commands:
```

<code>init</code>	Prepare your working directory for other commands
<code>validate</code>	Check whether the configuration is valid
<code>plan</code>	Show changes required by the current configuration
<code>apply</code>	Create or update infrastructure
<code>destroy</code>	Destroy previously-created infrastructure

```
...
```

```
Global options (use these before the subcommand, if any):
```

<code>-chdir=DIR</code>	Switch to a different working directory before executing the given subcommand.
<code>-help</code>	Show this help output, or the help for a specified subcommand.
<code>-version</code>	An alias for the "version" subcommand.

Terraform Code

- Terraform code is based on the [HCL2 toolkit](#). HCL stands for HashiCorp Configuration Language.
- Terraform code, or simply terraform is a declarative language that is specifically designed for provisioning infrastructure on any cloud or platform.
- Terraform code can be built incrementally, one or two resources at a time.

```
resource aws_vpc "main" {  
  cidr_block      = "10.0.0.0/16"  
  instance_tenancy = "dedicated"  
}
```

Terraform Comments

- **Line Comments** begin with an [octothorpe](#), or pound symbol: **#**

```
# This is a line comment.
```

- **Block comments** are contained between **/*** and ***/** symbols.

```
/* This is a block comment.  
   Block comments can span multiple lines.  
   The comment ends with this symbol: */
```

Command: fmt (Terraform Format)

- Terraform comes with a built in code formatter/cleaner. It can make all your margins and list indentation neat and tidy. Beauty works better.

```
$ terraform fmt
```

- Simply run it in a directory containing *.tf files and it will tidy up your code for you.

공급자(Provider) 구성

- 테라폼을 사용하는 첫 번째 단계는 공급자(provider) 구성하는 것입니다.
- The Terraform core program isn't very useful by itself. Terraform needs the help of a *provider* to be able to talk to cloud APIs.
- Terraform has hundreds of different providers.
- You can browse the provider list here:
 - <https://registry.terraform.io/browse/providers>
- We'll be using a few different providers, but the main one is the *aws* provider.

공급자 (Provider) 구성 (계속)

Providers

Providers are a logical abstraction of an upstream API. They are responsible for understanding API interactions and exposing resources.

Featured Providers

**AWS**
by  HashiCorp
[Docs](#) 

**Azure**
by  HashiCorp
[Docs](#) 

**Google Cloud Platform**
by  HashiCorp
[Docs](#) 

**Kubernetes**
by  HashiCorp
[Docs](#) 

<https://registry.terraform.io/browse/providers>

How to use this provider

To install this provider, copy and paste this code into your Terraform configuration. Then, run `terraform init`.

Terraform 0.13+

```
terraform {  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "6.58.0"  
    }  
  }  
}  
  
provider "aws" {  
  # Configuration options  
}
```

공급자(Provider) 구성 (계속)

Codes

- <https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/providers.tf>
- <https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/versions.tf>

```
terraform {  
  required_version = ">= 1.5.0, < 2.0.0"  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}
```

```
provider "aws" {  
  region = var.aws_region  
}
```

References

- <https://registry.terraform.io/browse/providers>
- <https://registry.terraform.io/providers/hashicorp/aws/latest>

Versioning Operators

- Re-usable modules should constrain only the minimum allowed version, such as `>= 2.35.0`.

- `=` (or no operator) : exact version equality
- `!=` : version not equal
- `>`, `>=`, `<`, `<=` : version comparison
- `~>` : Allows only the rightmost version component to increment. For example, to allow new patch releases within a specific minor release, use the full version number
 - `~> 1.0.4` will allow installation of 1.0.5 and 1.0.10 but not 1.1.0
 - `~> 0.9` is equivalent to `>= 0.9, < 1.0`
 - `~> 0.8.4` is equivalent to `>= 0.8.4, < 0.9`
 - This is usually called the [pessimistic constraint](#) operator.

- 참고자료: [Version Constraints](#)

Variables and Outputs

- Terraform 언어에는 명명된 값을 요청하거나 게시하기 위한 몇 가지 종류의 블록이 포함되어 있습니다.
 - 입력 변수([Input Variables](#))는 Terraform 모듈의 매개변수 역할을 하므로 사용자는 소스를 편집하지 않고도 동작을 사용자 정의할 수 있습니다. 함수의 입력 파라미터 (인자값)와 비슷한 개념입니다.
 - 출력 값([Output Values](#))은 Terraform 모듈의 반환 값과 같습니다. 함수의 리턴값과 비슷한 개념입니다.
 - 로컬 값([Local Values](#))은 식에 짧은 이름을 할당하기 위한 편리한 기능입니다. 함수의 임시 지역 변수와 비슷한 개념입니다.

입력 변수(variable) 정의 및 활용

- 입력 변수([Input Variable](#))를 사용하면 DRY(Don't Repeat Yourself) 원칙에 따라 코드가 중복되지 않도록 구성하기에 편리합니다. 변수를 선언하는 구문은 다음과 같습니다.

```
variable "NAME" {  
    [CONFIG ...]  
}
```

- 빈번하게 활용되는 3개의 매개 변수가 있습니다. 모두 선택적 매개 변수입니다.
 - [description](#) - 입력 변수의 사용 방법을 문서화하려면 이 변수에 설명을 기술합니다. `plan` 또는 `apply` 명령 실행시 이 설명을 볼 수 있습니다.
 - [default](#) - 변수의 기본(default) 값. 만일, 기본 값이 지정되어 있지 않으면 테라폼은 대화식으로 사용자에게 변수에 대한 정보를 묻습니다.
 - [type](#) - 변수의 타입을 지정. `string`(문자열), `number`(숫자), `bool`(불리언), `list`(리스트), `map`(맵), `set`(집합), `object`(객체), `tuple`(튜플) 등의 타입이 있습니다. 유형을 지정하지 않으면 테라폼은 `any`로 간주합니다.

Hands-on ([Ref.](#))

Terraform Working Directory로 이동합니다.

```
cd /home/ubuntu/terraform-iac-essential/terraform
terraform init
terraform fmt
terraform validate
```



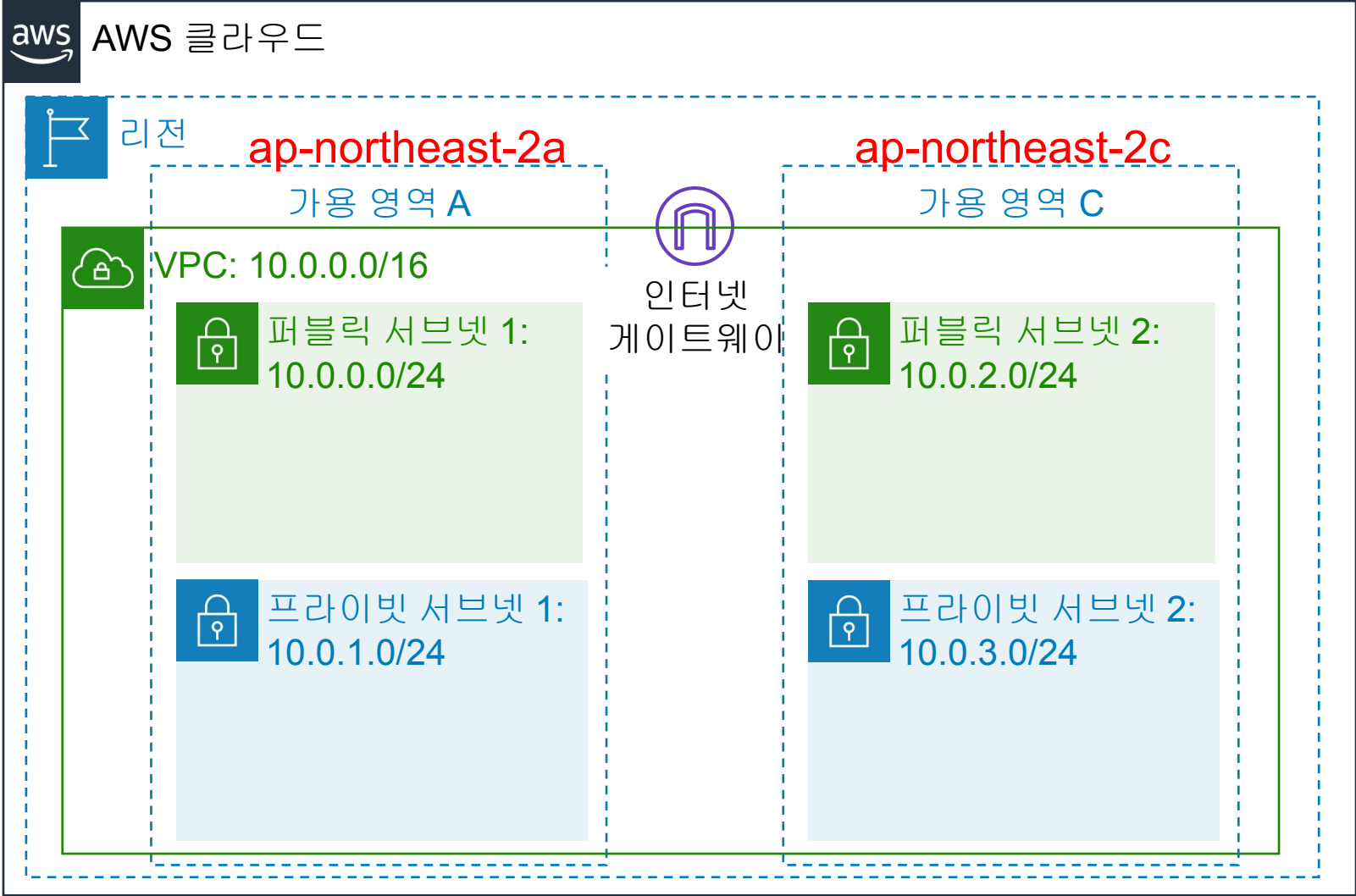
- `init` : AWS Provider를 내려받고 Working Directory 초기화
- `fmt` : Terraform 표준 형식으로 HCL 정렬
- `validate` : Syntax와 Resource Reference의 정적 오류 검사
- `plan` : 실제 Infrastructure 변경을 계산하여 사람이 검토
- `apply` : 검토한 Plan을 AWS에 적용

Custom VPC, Subnet 생성

- 일반적으로 기본 VPC(Default VPC)를 사용하는 것은 권장되지 않습니다.
- **Amazon Virtual Private Cloud([Amazon VPC](#))**를 사용하면 사용자가 정의한 가상 네트워크에서 **AWS(Amazon Web Services)** 리소스를 시작할 수 있습니다. 이 가상 네트워크는 고객의 자체 데이터 센터에서 운영하는 기존 네트워크와 매우 유사하며, **AWS**의 확장 가능한 인프라를 사용한다는 이점이 있습니다.
- 또한, 여러 가용 영역에 걸쳐 있는 **VPC**를 생성할 수 있습니다.

이 실습에서 다루는 주제

- 커스텀 VPC 및 서브넷을 생성합니다.



퍼블릭 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬
0.0.0.0/0	인터넷 게이트웨이

리소스(Resource) 구성

- 각 유형의 공급자(provider)마다 서버, 데이터베이스 및 로드 밸런서와 같은 다양한 종류의 리소스가 있습니다.
- 테라폼에서 리소스를 생성할 때는 다음과 같은 구문을 사용합니다.

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
    [CONFIG ...]  
}
```

- **<PROVIDER>**: aws와 같은 공급자의 이름, [Browse Providers | Terraform Registry](#)
- **<TYPE>**: [instance](#), [virtual_machine](#), [compute_instance](#) 같이 해당 공급자에서 생성할 리소스 유형
- **<NAME>**: 테라폼 코드에서 리소스를 참조할 때 사용하는 my_instance와 같은 식별자
- **[CONFIG ...]**: 특정 리소스에 대한 [하나 이상의 인수\(argument\)](#)들로 구성됨

변수의 보간(interpolation)

- 문자열 리터럴 내에서 참조를 사용하려면 보간(interpolation)을 사용합니다.

```
"${...}"  
"${var.server_port}"
```

- 참고자료: [Interpolation Syntax](#)
 - [simple math](#) in interpolations: `${count.index + 1}`
 - [conditionals](#) to determine a value
 - `count = "${var.something ? 1 : 0}"`
 - `subnet = "${var.env == "production" ? var.prod_subnet : var.dev_subnet}"`

리소스(Resource) 구성 (계속)

Codes

- <https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/vpc.tf>

Resources

- [aws_vpc](#)
- [aws_internet_gateway](#)
- [aws_subnet](#)
- [aws_route_table](#)
- [aws_route_table_association](#)

VPC, IGW 생성

- 생성된 VPC 및 인터넷 게이트웨이를 그림으로 표기하면 아래와 같습니다.



퍼블릭 서브넷 생성

- 추가로 생성할 퍼블릭 서브넷 구성을 자세히 그림으로 표기하면 아래와 같습니다.

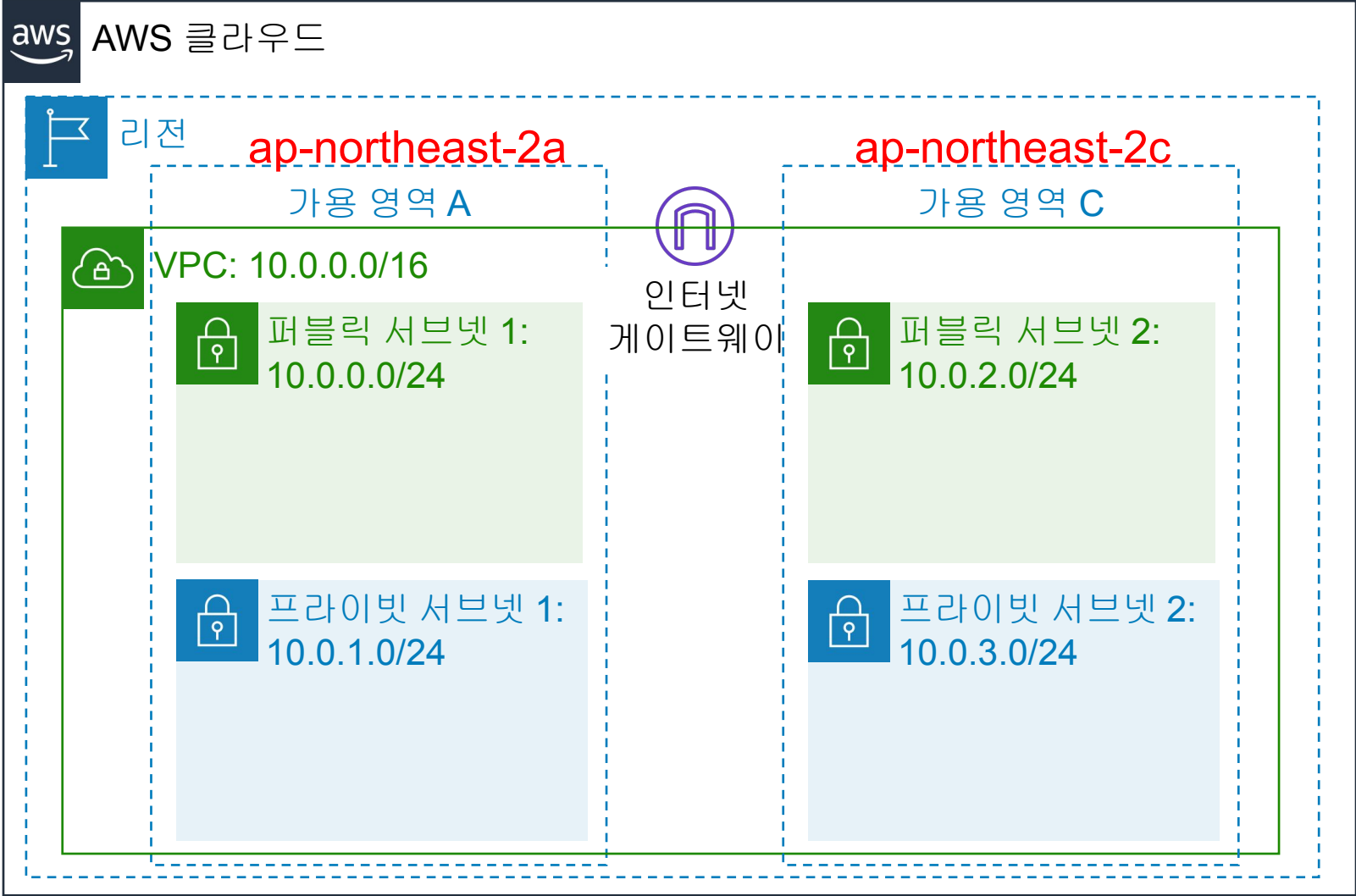


퍼블릭 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬
0.0.0.0/0	인터넷 게이트웨이

프라이빗 서브넷 생성

- 생성할 프라이빗 서브넷 구성을 자세히 그림으로 표기하면 아래와 같습니다.



퍼블릭 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬
0.0.0.0/0	인터넷 게이트웨이

프라이빗 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬

2-1. Network Bootstrap

Golden Image Builder를 실습 VPC의 Public Subnet에 배치하기 위해 최초 한 번만 Network를 생성합니다.

```
terraform plan \
  -target=aws_vpc.main \
  -target=aws_internet_gateway.main \
  -target=aws_subnet.pub_a \
  -target=aws_subnet.pub_c \
  -target=aws_subnet.pri_a \
  -target=aws_subnet.pri_c \
  -target=aws_route_table.pub \
  -target=aws_route_table_association.pub_a \
  -target=aws_route_table_association.pub_c

terraform apply \
  -target=aws_vpc.main \
  -target=aws_internet_gateway.main \
```



Changing Existing Infrastructure

- Whenever you run a plan or apply, Terraform reconciles three different data sources:
 1. What you wrote in your code
 2. The state file
 3. What actually exists
- Terraform does its best to add, delete, change, or replace existing resources based on what is in your *.tf files. Here are the four different things that can happen to each resource during a plan/apply:

```
+   create
-   destroy
-/+  replace (또는 +/- replace)
~   update in-place
```

테라폼 상태 (Terraform State)

- 테라폼은 구성 파일을 위해 생성된 모든 리소스를 추적합니다.
- 테라폼은 상태 파일을 이용하여 리소스가 이미 존재한다는 것을 알고 있습니다.
- 그래서 앞에서 **apply** 명령어를 실행할 때 아래와 같이 **‘Refreshing state...’** 라는 메시지를 나타냅니다. 또한, 현재 배포되어 있는 인프라의 내용과 테라폼 코드의 내용이 어떻게 다른지도 보여주었습니다.

```
tf-user:~/environment $ terraform apply
aws_instance.web: Refreshing state... [id=i-0254b858249412bef]

Note: Objects have changed outside of Terraform

Terraform detected the following changes made outside of Terraform since the last "terraform apply":

# aws_instance.web has changed
~ resource "aws_instance" "web" {
    id                = "i-0254b858249412bef"
    + tags             = {}
    # (29 unchanged attributes hidden)
```

Command: state

```
$ terraform state  
$ terraform state list  
$ terraform state show aws_instance.web
```

- Advanced state management

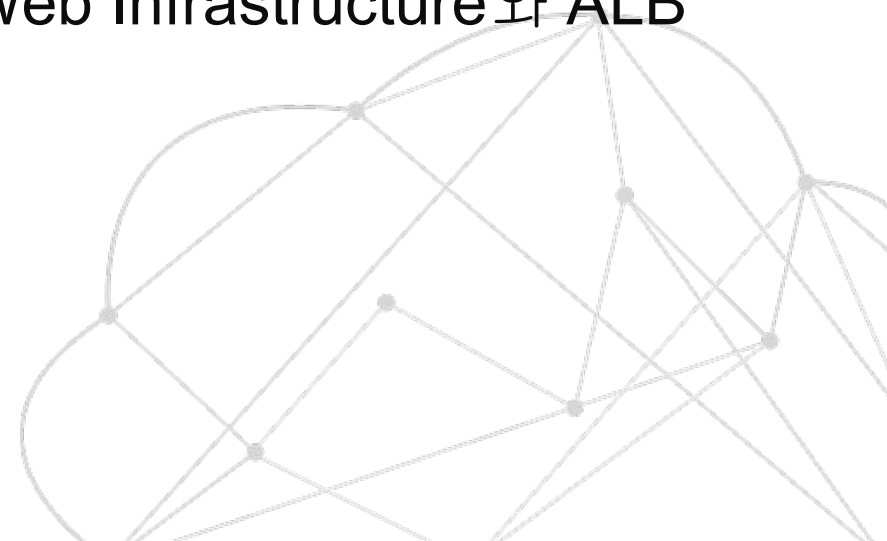
Subcommands:

list	List resources in the state
mv	Move an item in the state
pull	Pull current state and output to stdout
push	Update remote state from a local state file
replace-provider	Replace provider in the state
rm	Remove instances from the state
show	Show a resource in the state

SUB CHAPTER

1. Custom VPC 기반 Highly Available Web Architecture 구축

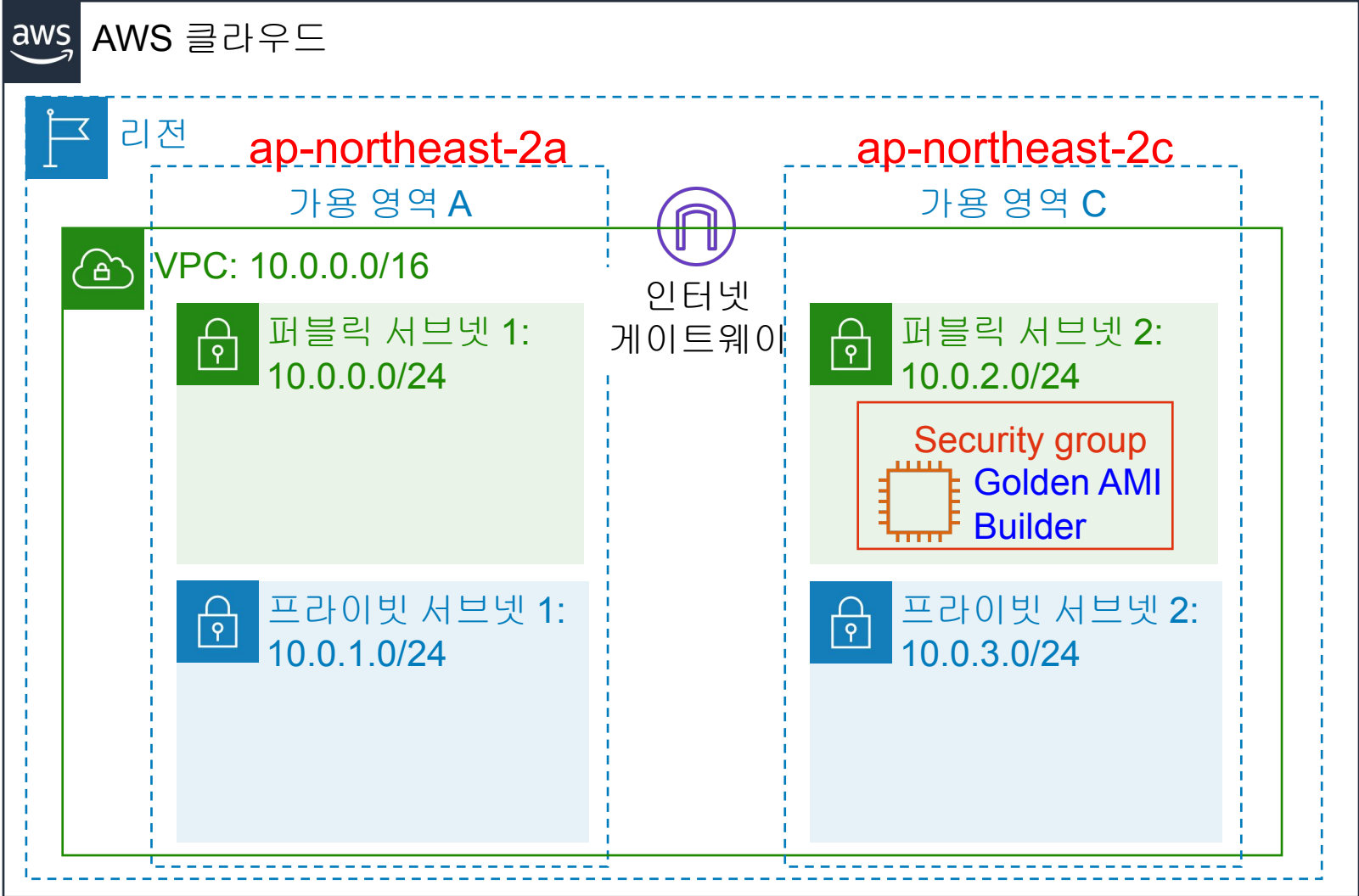
- Lab 1. Development Environment
- Lab 2. Terraform Basics와 Network
- Lab 3. Golden AMI
- Lab 4. Private Web Infrastructure와 ALB



Lab 3. Golden AMI

이 실습에서 다루는 주제

- AWS 관리콘솔에서 Golden Image Builder EC2를 구성하고 AMI 이미지를 생성합니다.



퍼블릭 라우팅 테이블

목적지	대상
10.0.0.0/16	로컬
0.0.0.0/0	인터넷 게이트웨이

Lab 3. Golden AMI

Golden Image Builder EC2는 Terraform으로 만들지 않습니다.

AWS Console

- Public Subnet C
- Golden Image Builder EC2
- userdata/image-builder.sh
- Web Server 확인
- AMI 생성
- Golden AMI ID
- terraform.tfvars 수정



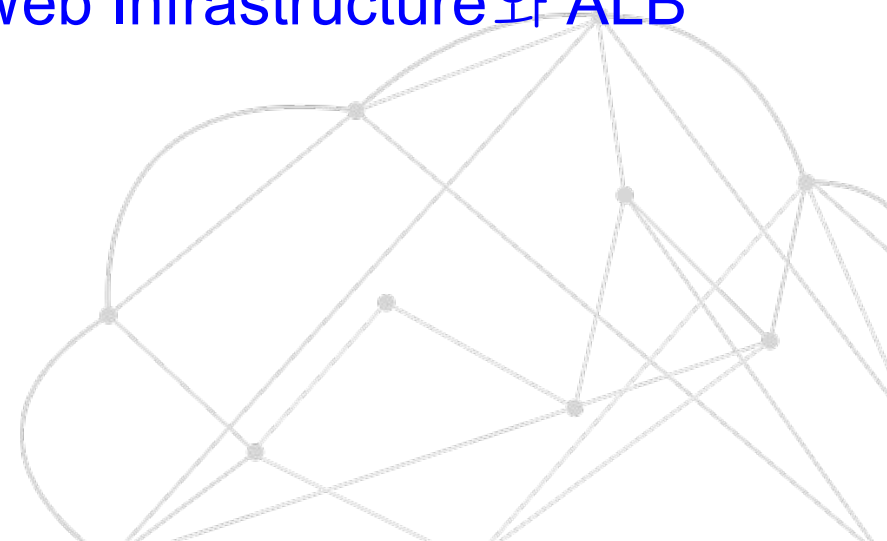
EC2 → Instances → Launch instances:

- Name: terraform-iac-essential-image-builder
- AMI: Amazon Linux 2023 x86_64

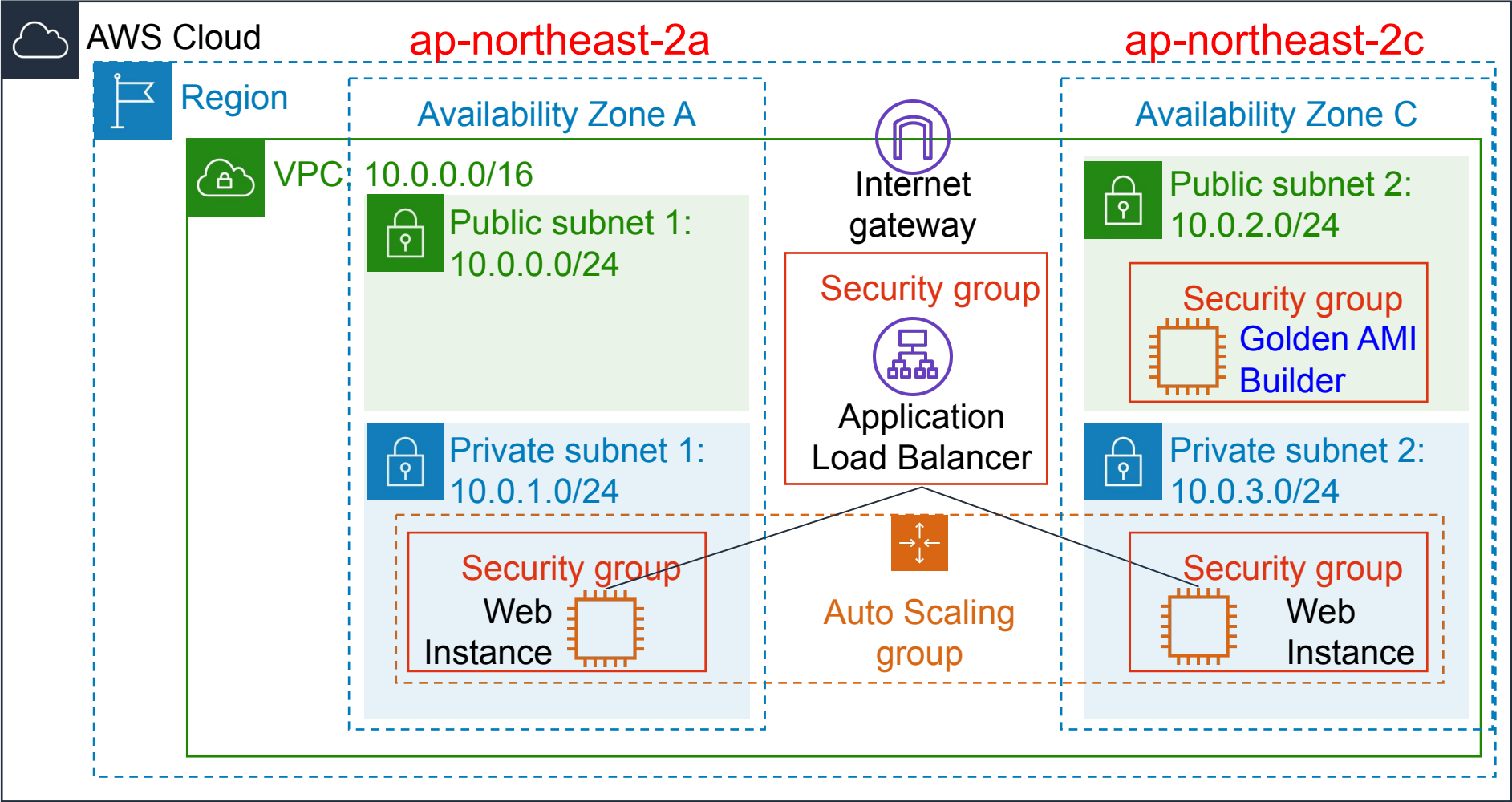
SUB CHAPTER

1. Custom VPC 기반 Highly Available Web Architecture 구축

- Lab 1. Development Environment
- Lab 2. Terraform Basics와 Network
- Lab 3. Golden AMI
- Lab 4. Private Web Infrastructure와 ALB



Lab 4. Private Web Infrastructure와 ALB



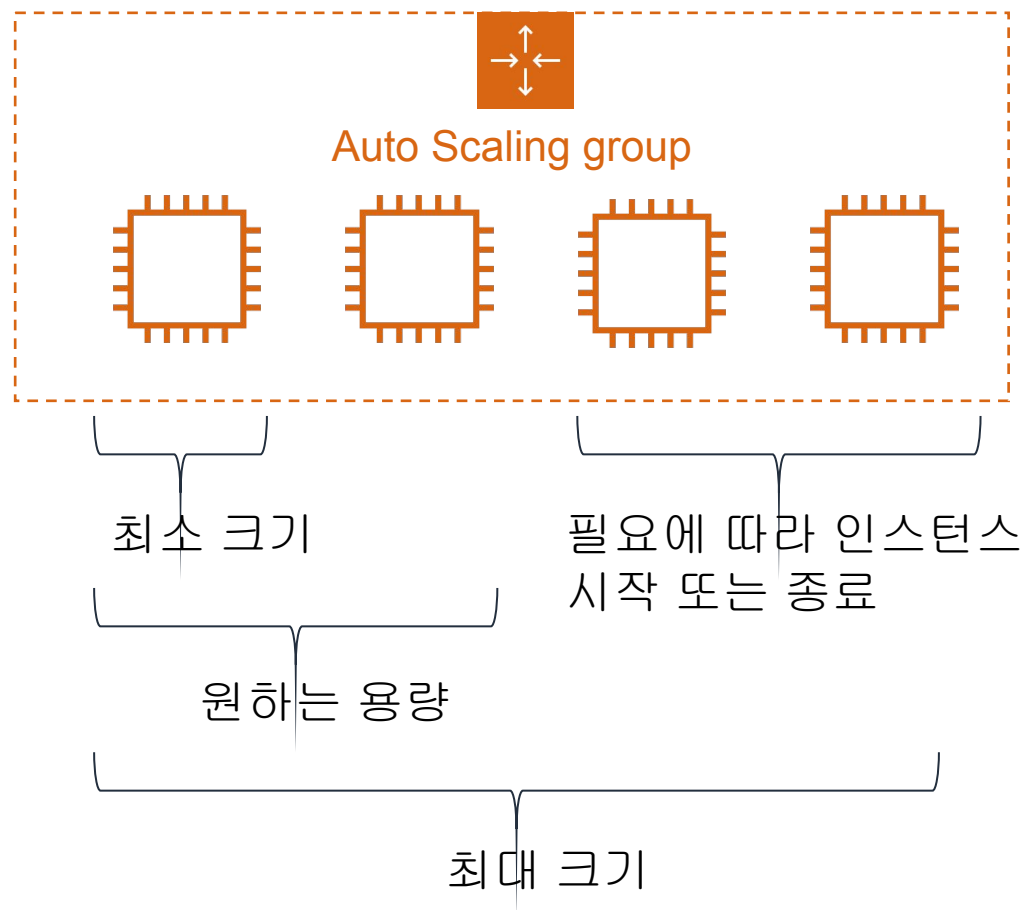
Codes

- https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/security_groups.tf

Resources

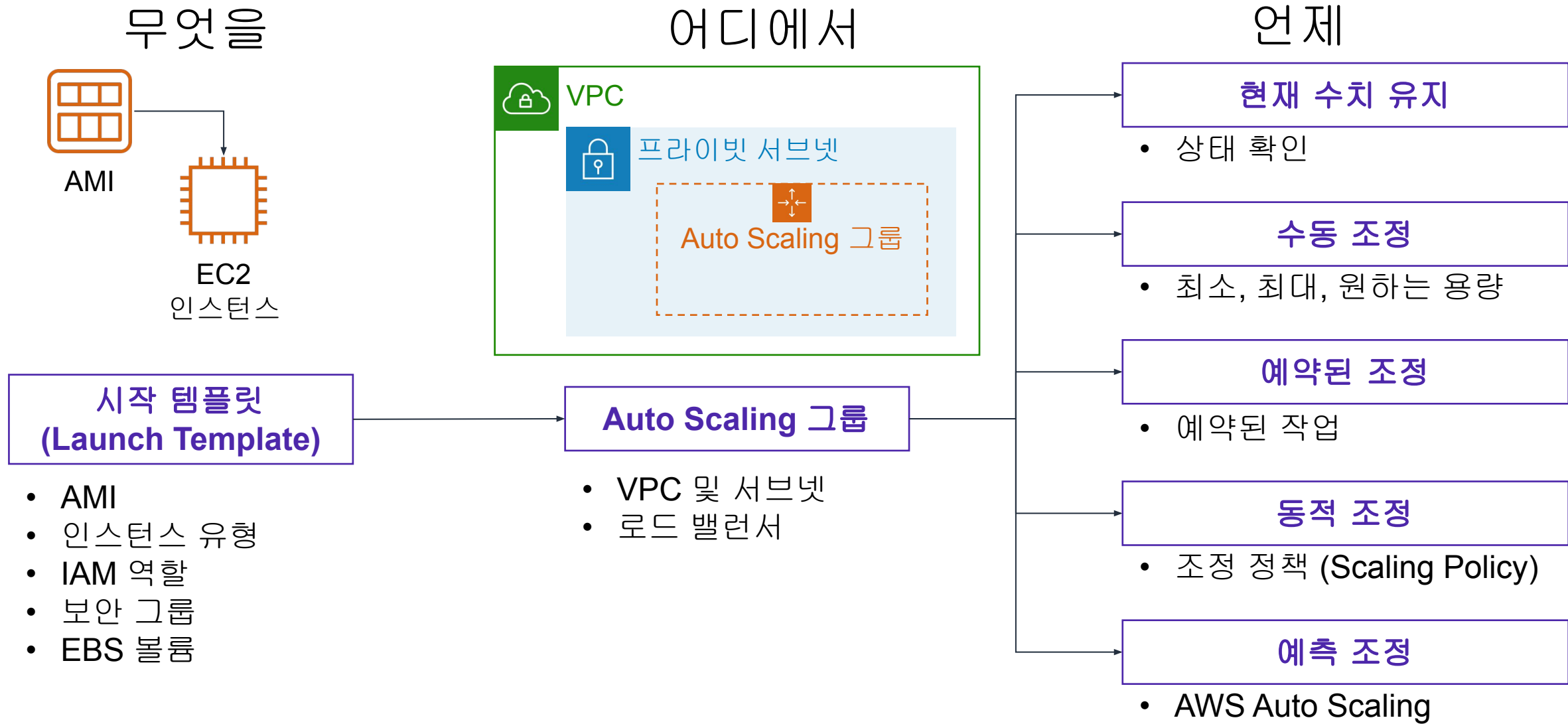
- [aws_security_group](#)

Auto Scaling 그룹



- Auto Scaling 그룹은 자동 조정 및 관리 목적의 논리적 그룹으로 다뤄지는 EC2 인스턴스의 모음입니다.

Amazon EC2 Auto Scaling의 작동 방식 (중요!)



Private Web Infrastructure

Codes

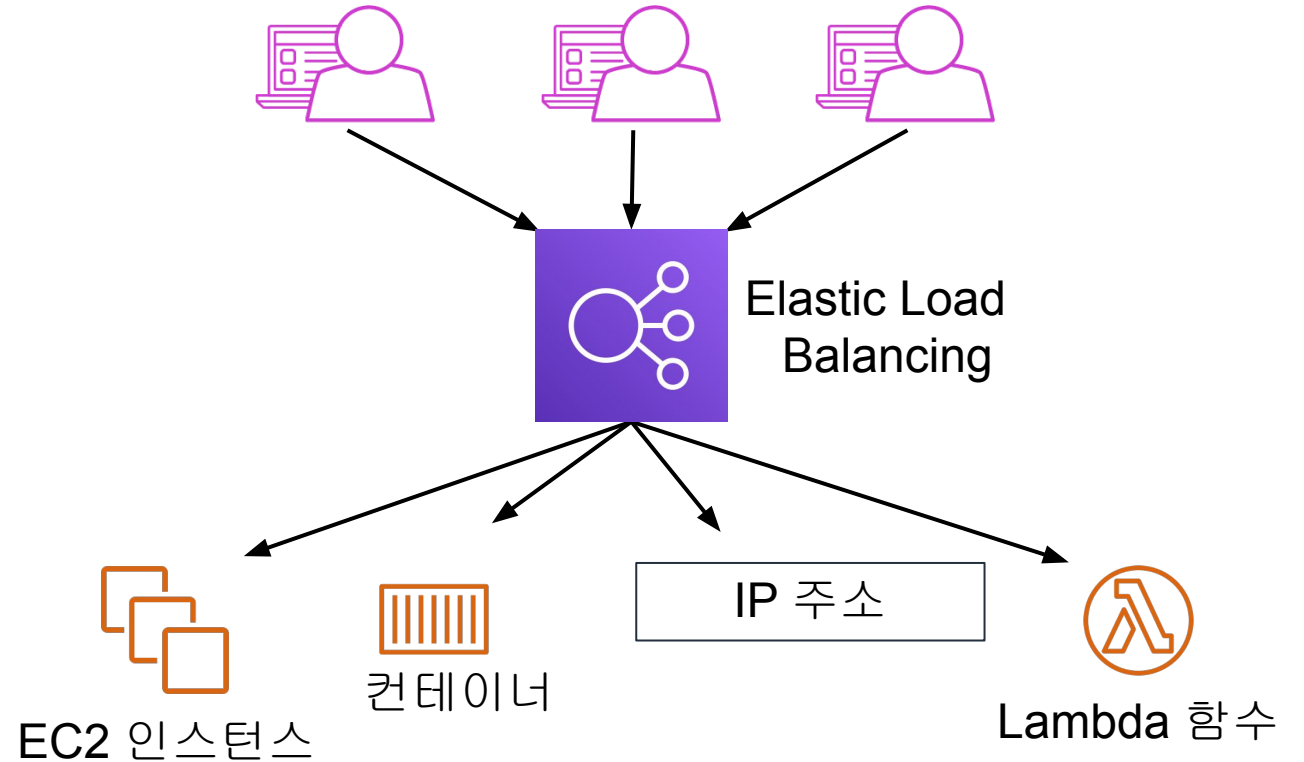
- <https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/asg.tf>

Resources

- [aws_launch_template](#)
- [aws_autoscaling_group](#)

탄력적 로드 밸런싱 (Elastic Load Balancing)

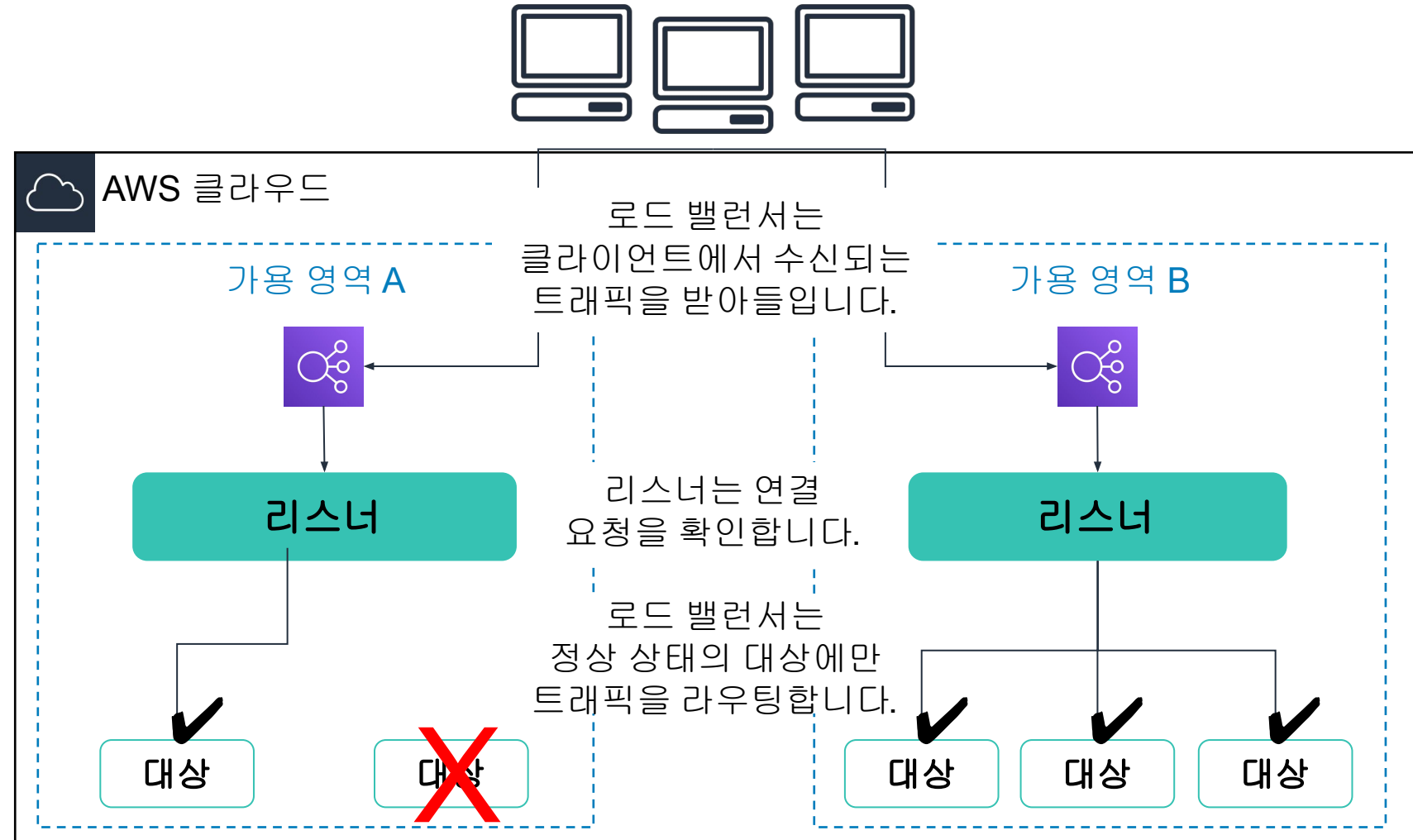
- 수신되는 애플리케이션 또는 네트워크 트래픽을 단일 가용 영역 또는 여러 가용 영역의 여러 대상에 분산합니다.
- 시간이 지나면서 애플리케이션 트래픽의 변화에 따라 로드 밸런서가 자동으로 확장(Scale-out) 및 축소(Scale-in) 됩니다.
- 로드 밸런서는 **AWS**가 관리하는 완전관리형서비스(Fully Managed Service)



Elastic Load Balancing의 작동 방식 (중요!)

- Application Load Balancer 및 Network Load Balancer를 사용할 때는 대상 그룹(Target Group)에 대상(Target)을 등록하고 대상 그룹으로 트래픽을 라우팅합니다.
- Classic Load Balancer를 사용하는 경우 로드 밸런서에 인스턴스를 등록합니다.

로드 밸런서는 상태 확인을 수행하여 등록된 대상의 상태를 모니터링합니다.



Private Web Infrastructure와 ALB

Codes

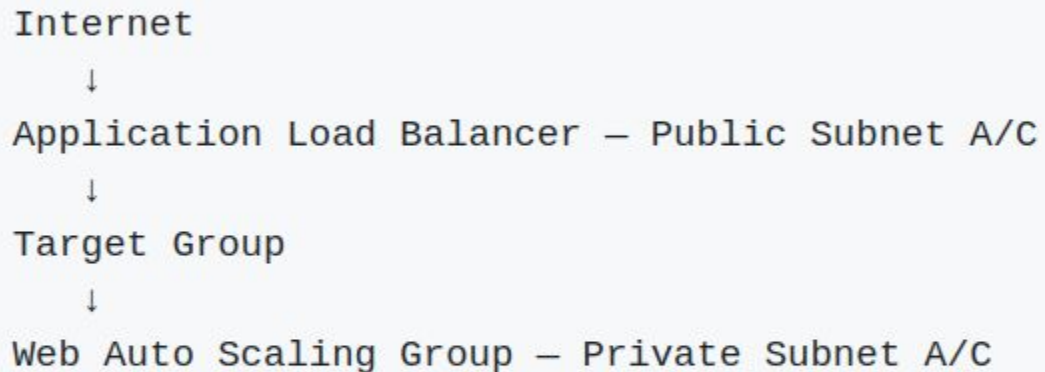
- https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/load_balancer.tf
- <https://github.com/seungsuk-training/terraform-iac-essential/blob/main/terraform/outputs.tf>

Resources

- [aws_lb_target_group](#)
- [aws_lb](#)
- [aws_lb_listener](#)

Lab 4. Private Web Infrastructure와 ALB

Source에서 다음 관계를 확인합니다.



```
graph TD; Internet --> ALB[Application Load Balancer - Public Subnet A/C]; ALB --> TG[Target Group]; TG --> WASG[Web Auto Scaling Group - Private Subnet A/C];
```

Internet

↓

Application Load Balancer – Public Subnet A/C

↓

Target Group

↓

Web Auto Scaling Group – Private Subnet A/C

ALB는 Public Entry Point, Load Distribution, Health Check와 Multi-AZ High Availability를 제공합니다. Web EC2는 Public IP 없이 Private Subnet에 배치되고 ALB Security Group에서 오는 HTTP만 허용합니다.

확인할 파일:



AI를 활용한 Terraform Code 분석 및 개선

AI를 활용한 Terraform Code 분석 및 개선

Lab 5. Kiro CLI Hands-on

Kiro CLI 개요

- Kiro CLI는 터미널에서 자연어 기반으로 코드 생성, 인프라 구성, 배포 자동화를 수행하는 AI 기반 개발 도구
 - 자연어로 프롬프트를 입력 → AI가 이를 해석해 실행까지 연결
 - CLI(터미널) 환경에서 AI 에이전트와 대화하듯 기능을 구현·배포
 - 인프라 구성, 코드 생성, 배포 자동화, 작업 분해(Tasking) 등을 자연어 기반으로 처리
- 왜 Kiro CLI가 중요한가?
 - 전통적인 DevOps/Infra-as-Code는 스크립트, 템플릿, 수동 설정의 반복이 많음
 - Kiro CLI는 자연어 → 실행 가능한 코드/infra 템플릿 → 실제 배포까지 이어주는 Conversational Infrastructure를 제공
 - 결과적으로 반복적인 구성 작업 감소, 오류 감소, 그리고 생산성 대폭 향상을 기대할 수 있음

Lab 5. Kiro CLI Hands-on — 15~20분

지금까지 직접 구축한 Terraform Infrastructure를 AI가 분석하고 개선하도록 요청해 봅니다.

Kiro CLI는 Local Windows/macOS나 별도 EC2, WSL2 또는 AWS CloudShell이 아니라 모든 교육生在 공통으로 사용하는 Ubuntu Development EC2에서 실행합니다. 긴 Prompt 전문은 [ai/prompts.md](#) 를 열어 Copy & Paste합니다.

AI가 Terraform과 Architecture를 대신 이해해 주는 것은 아닙니다. Terraform과 Architecture를 이해한 사람이 AI를 활용하면 코드 분석, 생성, Refactoring 및 검증 속도를 높일 수 있습니다. AI가 생성하거나 수정한 Infrastructure Code를 바로 적용하지 않고 반드시 `terraform validate` 와 `terraform plan` 결과를 사람이 검토합니다.

5-1. Kiro CLI 로그인

환경 설정 Script에서 설치한 Version을 확인하고 Remote-SSH 환경에 적합한 Device Flow로 로그인합니다.

Kiro CLI 기반 인프라 구축 실습 (계속)

[참고] 현재 크레딧 사용량과 잔액을 조회

- /usage

```
12% > /usage
```

Estimated Usage | resets on 2026-04-01 | **KIRO FREE**

Credits (3.22 of 50 covered in plan)

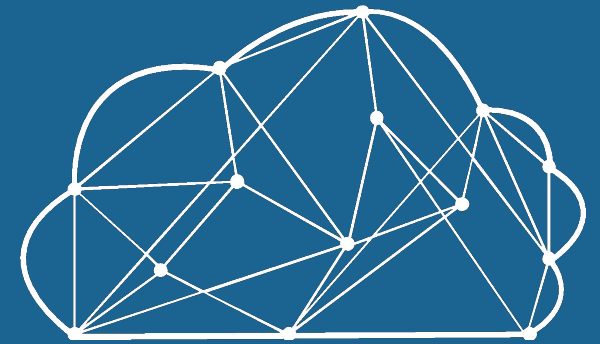


Overages: **Disabled**

To manage your plan or configure overages navigate to <https://app.kiro.dev/account/usage>

Tip: to see context window usage, run `/context`

12% >



리소스 삭제

리소스 삭제

Lab 6. Cleanup — 비용 방지 필수

6-1. Golden Image Builder 먼저 삭제

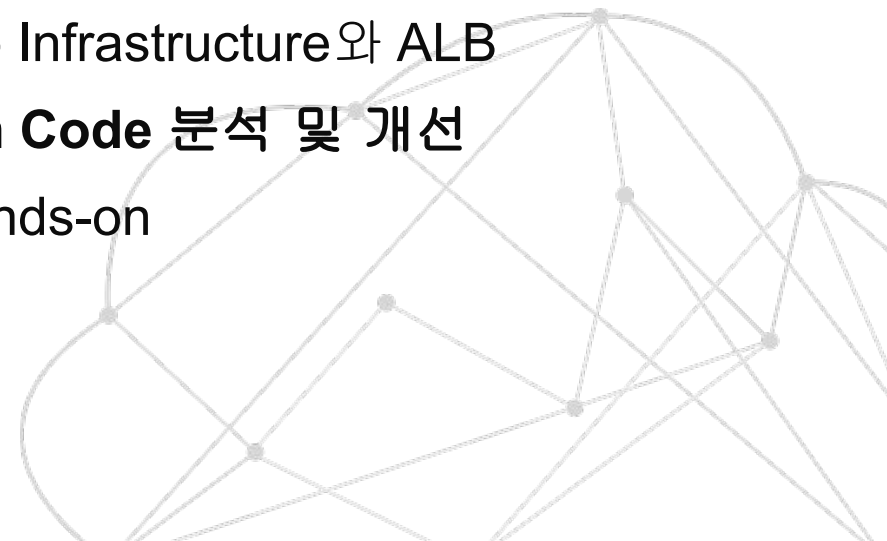
Golden Image Builder EC2와 `WEB-SG` 는 Terraform이 생성한 Custom VPC 안에 있지만 Terraform State 밖에서 수동으로 만들었습니다. 이 Resource가 남아 있으면 Terraform이 Subnet 또는 VPC를 삭제할 때 Dependency 오류가 발생할 수 있으므로 `terraform destroy` 보다 먼저 정리합니다.

AWS Console에서:

1. EC2 → Instances → `terraform-iac-essential-image-builder` 선택
2. Instance state → Terminate instance
3. Instance 상태가 `Terminated` 가 될 때까지 대기
4. EC2 → Security Groups → `WEB-SG` 선택
5. 연결된 Network Interface가 없는지 확인한 후 `WEB-SG` 삭제

Summary and Q & A

1. Terraform을 이용한 Infrastructure as Code 이해
2. IAM 계정 생성 및 Default VPC 확인
3. Custom VPC 기반 Highly Available Web Architecture 구축
 - Lab 1. Development Environment
 - Lab 2. Terraform Basics와 Network
 - Lab 3. Golden AMI
 - Lab 4. Private Web Infrastructure와 ALB
4. AI를 활용한 Terraform Code 분석 및 개선
 - Lab 5. Kiro CLI Hands-on
5. 리소스 삭제





THANK YOU.

BESPIN GLOBAL